

Graphenbasierte DNA-Sequenzierung mittels des Subgraph Algorithmus

Stephan Epp

5. April 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Beitrag dieser Arbeit	3
1.2	Problemstellung	4
2	Grundlagen	5
2.1	Biologische Grundlagen der DNA-Struktur	5
2.2	Definitionen	5
2.3	Signatur-Funktion	6
3	Der Subgraph-DNA-Algorithmus	7
3.1	Kernidee	7
3.2	Algorithmus-Beschreibung	7
3.3	Implementierung in Python	9
4	Formale Analyse	10
4.1	Korrektheit	10
4.2	Laufzeit	11
4.3	Optimalität der Laufzeit	12
5	Anwendungen in der DNA-Analyse	15
5.1	Mutationsanalyse	15
5.2	Vergleichende Genomik und phylogenetische Analyse	16
5.3	Sequenzierungspipelines (NGS)	16
5.4	Statistische Eigenschaften von DNA als Graph	17
6	Metagenomik: Subgraph-Analyse heterogener Genomproben	17
6.1	Biologische Grundlagen der Metagenomik	17
6.2	Formale Modellierung: Metagenom-Graph	19
6.3	Subgraph-basierte Read-Deduplizierung	20
6.4	Untere Schranke für metagenomische Deduplizierung	22
6.5	Taxonomische Klassifikation via LCS-Distanzmatrix	23
6.6	Vollständiges Beispiel: Metagenomische Analyse einer Bodenprobe	25

7	Fehlertolerante Subgraph-Analyse	25
7.1	Motivation: Sequenzierfehler in der Praxis	25
7.2	Fehlermodell und Bitfehler-Analyse	27
7.3	Der ε -Subgraph-Algorithmus	28
7.4	Korrektheit unter Fehlertoleranz	29
7.5	Falsch-Positiv-Rate und Präzisions-Analyse	30
7.6	Insertions- und Deletionsfehler	32
7.7	Kombiniertes Fehlermodell und optimale Schwellwahl	33
7.8	Laufzeit des ε -Subgraph-Algorithmus	35
7.9	Vollständiges Beispiel: Fehlertolerante Analyse	35
8	Zusammenfassung	36
8.1	Ausblick	38
8.1.1	Parallelisierung und Hardware-Beschleunigung	38
8.1.2	Gewichtete DNA-Graphen und Bindungsstärken	39
8.1.3	RNA-Sequenzanalyse und Sekundärstrukturen	39
8.1.4	Epigenetische Modellierung	39
8.1.5	Proteomik: Aminosäure-Graphen	40
8.1.6	Approximationsalgorithmen und PTAS	40
8.1.7	Quantenalgorithmen für die Rotationssuche	40
8.1.8	Fehlertoleranz: Weiterführende Untersuchungen	41
8.1.9	Integration in bestehende Bioinformatik-Pipelines	41
8.1.10	Erweiterung auf nicht-kanonische Nukleobasen	41
8.1.11	Phylogenetische Netzwerke und evolutionäre Distanzen	42
8.1.12	Anwendung auf nicht-DNA-Sequenzen	42
8.2	Implementierungshinweis	43

1 Einführung

Die effiziente Entschlüsselung und Analyse genetischer Sequenzen gehört zu den fundamentalen Herausforderungen der modernen Bioinformatik. Die vorliegende Arbeit stellt einen neuartigen Ansatz vor, der den Subgraph Algorithmus [1] zur strukturellen Analyse von DNA-Sequenzen nutzbar macht. Dabei werden DNA-Stränge als gerichtete Graphen modelliert: Nukleobasen bilden Knoten, kovalente Backbone-Bindungen sowie Wasserstoffbrücken zwischen Basenpaaren bilden gerichtete Kanten. Der Subgraph-basierte Vergleich ermöglicht es, genomische Teilsequenzen durch zyklische Rotation und Signaturvergleich in Zeit $O(n^3)$ zu erkennen, was gegenüber naiven Permutationsansätzen mit Komplexität $O(n! \cdot n^2)$ eine exponentielle Beschleunigung darstellt. Wir beweisen Korrektheit und Optimalität des Verfahrens formal, geben ein vollständiges Anwendungsbeispiel und diskutieren Implikationen für NGS-Sequenzierungspipelines, Mutationsanalyse und vergleichende Genomik.

Die Desoxyribonukleinsäure (DNA) ist der molekulare Träger der Erbinformation aller bekannten Lebewesen. Seit der Aufklärung der Doppelhelix-Struktur durch Watson und Crick (1953) hat die DNA-Sequenzierung einen dramatischen Wandel erfahren: Von der langwierigen Sanger-Sequenzierung über das Hochdurchsatz-Sequenzieren (NGS) bis hin zu mobilen Nanoporen-Geräten.

Ungeachtet dieser instrumentellen Fortschritte stellt der *algorithmische Vergleich* genomischer Sequenzen nach wie vor eine zentrale Herausforderung dar. Die Frage, ob ein DNA-Fragment S_A in einem längeren Fragment S_B als Teilsequenz enthalten ist — und wie man dies effizient prüft — ist zentral für Diagnose genetischer Erkrankungen, phylogenetische Analysen und die Erkennung von Mutationen.

1.1 Beitrag dieser Arbeit

Wir übertragen den Subgraph Algorithmus [1] auf die DNA-Analyse und erweitern ihn zu einem vollständigen Framework für strukturelle Sequenzanalyse und metagenomische Probenverarbeitung. Die Beiträge dieser Arbeit im Überblick:

- **Graphmodellierung (Kapitel 2):** DNA-Sequenzen werden als gerichtete Graphen G_{DNA} formalisiert. Adjazenzmatrizen kodieren Backbone- und Wasserstoffbrückenbindungen; eine injektive Spalten-Signatur ermöglicht effizienten Strukturvergleich (Lemma 1).
- **Algorithmus (Kapitel 3):** Der Subgraph-DNA-Algorithmus erkennt Teilsequenzen durch LCS-Vergleich auf Spalten-Signaturen. Zyklische Rotation reduziert $n!$ mögliche Permutationen auf n zu prüfende Fälle.
- **Korrektheit und Laufzeit (Kapitel 4):** Vollständigkeit und Präzision des Algorithmus werden formal bewiesen (Satz 1). Die Gesamtkomplexität beträgt $\Theta(n^3)$ (Satz 2).
- **Optimalität unter SETH (Kapitel 4):** Unter der Strong Exponential Time Hypothesis ist kein Algorithmus mit $O(n^{3-\epsilon})$ möglich (Satz 3) — der Algorithmus ist asymptotisch optimal.
- **Anwendungen (Kapitel 5):** Mutationsanalyse, NGS-Sequenzierungspipelines (De-novo-Assemblierung, Varianten-Calling, Redundanz-Elimination) und phylogenetische Analyse werden behandelt.

- **Metagenomik (Kapitel 6):** Für heterogene Genomproben mit M Reads werden entwickelt:
 - eine Subgraph-Halbordnung $\preceq_{\text{rot}}$ auf dem Read-Pool (Satz 4),
 - ein Deduplizierungsalgorithmus zur Berechnung des maximalen Kerngenoms $\mathcal{M}$ mit Laufzeit $O(M \cdot |\mathcal{M}| \cdot n^3)$ (Sätze 5, 6),
 - eine LCS-Pseudometrik d_{LCS} für taxonomische Distanzen (Lemma 7) und
 - ein Separierungssatz für korrekte Read-Zuordnung zu Organismen (Satz 8).
- **Fehlertolerante Subgraph-Analyse (Kapitel 7):** Reale Sequenzierdaten sind fehlerbehaftet. Dieses Kapitel entwickelt ein vollständiges formales Framework für fehlertolerantes Subgraph-Matching unter realitätsnahen Fehlermodellen:
 - eine Bitfehler-Analyse, die quantifiziert, wie viele Adjazenzmatrix-Einträge sich unter e Substitutionen ändern (Lemma 8),
 - der ε -Subgraph-Algorithmus mit technologie-spezifischem Toleranzparameter und Korrektheitsbeweis (Satz 9),
 - eine formale Analyse der Falsch-Positiv-Rate mit Chernoff-Abschätzung (Satz 10, Korollar 5),
 - eine Indel-Fehleranalyse mit Erkennungsgarantie (Satz 11), sowie
 - ein kombiniertes Fehlerbudget-Modell mit optimaler Schwellwahl $\varepsilon^*(\mathcal{T})$ für Illumina, PacBio HiFi und Nanopore (Satz 12).
- **Ausblick (Kapitel 8.5):** Erweiterungen auf gewichtete Graphen, RNA-Sekundärstrukturen, Epigenetik, Proteomik, Approximationsalgorithmen, Quantenalgorithmen (Speedup auf $O(n^{2.5})$ unter Grover) und Parallelisierung ($O(n^2)$ bei $P = n$ Prozessoren) werden diskutiert.

1.2 Problemstellung

Gegeben seien zwei DNA-Sequenzen $S_A = (a_1, a_2, \dots, a_{n_A})$ und $S_B = (b_1, b_2, \dots, b_{n_B})$ mit $a_i, b_j \in \{A, T, G, C\}$.

Ziel: Bestimme, ob die Graph-Repräsentation G_A ein Subgraph der Graph-Repräsentation G_B ist, unter Berücksichtigung zyklischer Sequenz-Rotationen.

Ergebnis:

- Wenn $G_B \supseteq G_A$: Fragment A ist in Fragment B enthalten — behalte G_B als informationsreichere Repräsentation.
- Wenn $G_A \supseteq G_B$: Fragment B ist Teilsequenz von A — behalte G_A .
- Wenn beide äquivalent: beliebiges Fragment behalten.
- Wenn keine Inklusion: beide Fragmente getrennt weiterführen.

Abbildung 1: DNA als gerichteter Graph

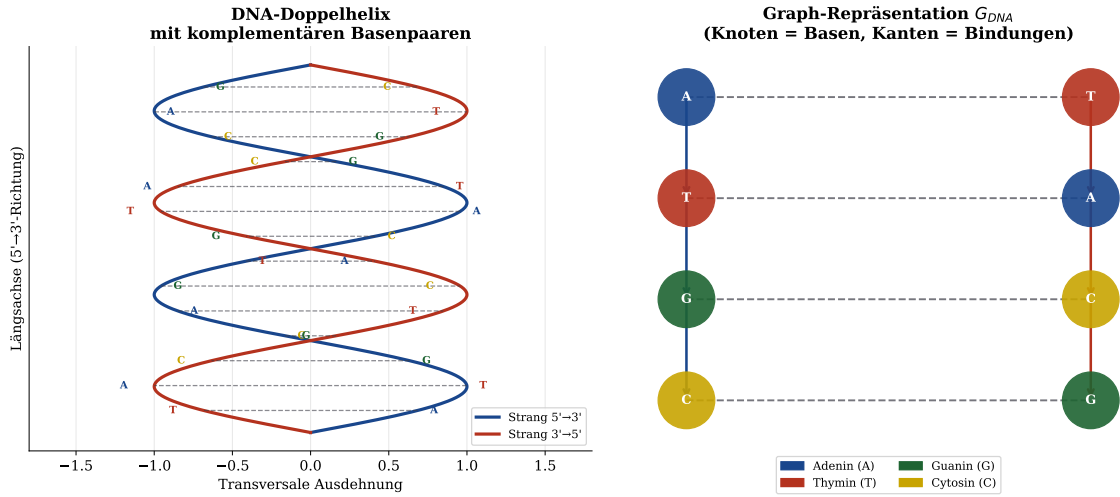


Abbildung 1: DNA-Doppelhelix und ihre Repräsentation als gerichteter Graph G_{DNA} . Knoten repräsentieren Nukleobasen (A = Adenin, T = Thymin, G = Guanin, C = Cytosin), Pfeile entlang des Backbones zeigen die $5' \rightarrow 3'$ -Leserichtung, gestrichelte Linien symbolisieren Wasserstoffbrückenbindungen zwischen komplementären Strängen.

2 Grundlagen

2.1 Biologische Grundlagen der DNA-Struktur

Die DNA besteht aus zwei antiparallelen Polynukleotidsträngen, die durch Wasserstoffbrücken zwischen komplementären Basenpaaren (A–T und G–C) zur Doppelhelix zusammengehalten werden. Für die graphentheoretische Modellierung sind drei Klassen von Bindungen relevant:

- **Backbone-Bindungen:** Kovalente Phosphodiesterbindungen zwischen aufeinanderfolgenden Nukleotiden entlang des $5' \rightarrow 3'$ -Strangs.
- **Watson-Crick-Bindungen:** Wasserstoffbrücken zwischen A und T (zwei Bindungen) sowie G und C (drei Bindungen).
- **Stacking-Wechselwirkungen:** π - π -Wechselwirkungen zwischen benachbarten Basen (in dieser Arbeit nicht gesondert modelliert).

2.2 Definitionen

Definition 1 (DNA-Alphabet). Das DNA-Alphabet ist definiert als $\Sigma_{DNA} = \{A, T, G, C\}$, wobei A für Adenin, T für Thymin, G für Guanin und C für Cytosin steht. Die Komplementaritätsrelation ist festgelegt durch: $\overline{A} = T$, $\overline{T} = A$, $\overline{G} = C$, $\overline{C} = G$.

Definition 2 (DNA-Graph). Sei $S = (s_1, s_2, \dots, s_n)$ eine DNA-Sequenz mit $s_i \in \Sigma_{DNA}$. Der DNA-Graph $G_S = (V, E)$ ist definiert durch:

$$\begin{aligned} V &= \{v_1, v_2, \dots, v_n\} \quad (\text{je ein Knoten pro Nukleotid}) \\ E &= E_{\text{back}} \cup E_{\text{wc}} \end{aligned}$$

mit Backbone-Kanten $E_{\text{back}} = \{(v_i, v_{i+1}) \mid 1 \leq i < n\}$ und Watson-Crick-Kanten

$$E_{\text{wc}} = \{(v_i, v_j) \mid j \neq i, s_j = \overline{s_i}\}.$$

Definition 3 (DNA-Adjazenzmatrix). Die Adjazenzmatrix $A^S \in \{0, 1\}^{n \times n}$ des DNA-Graphen G_S ist:

$$A_{ij}^S = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E \\ 0 & \text{sonst} \end{cases}$$

Definition 4 (Subgraph). Ein Graph $G = (V, E)$ ist ein Subgraph von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$.

Definition 5 (Zyklische Rotation). Eine zyklische Rotation der Sequenz $[a_0, a_1, \dots, a_{n-1}]$ um k Positionen nach rechts ist definiert als:

$$\text{rotate}_k([a_0, a_1, \dots, a_{n-1}]) = [a_k, a_{k+1}, \dots, a_{n-1}, a_0, \dots, a_{k-1}]$$

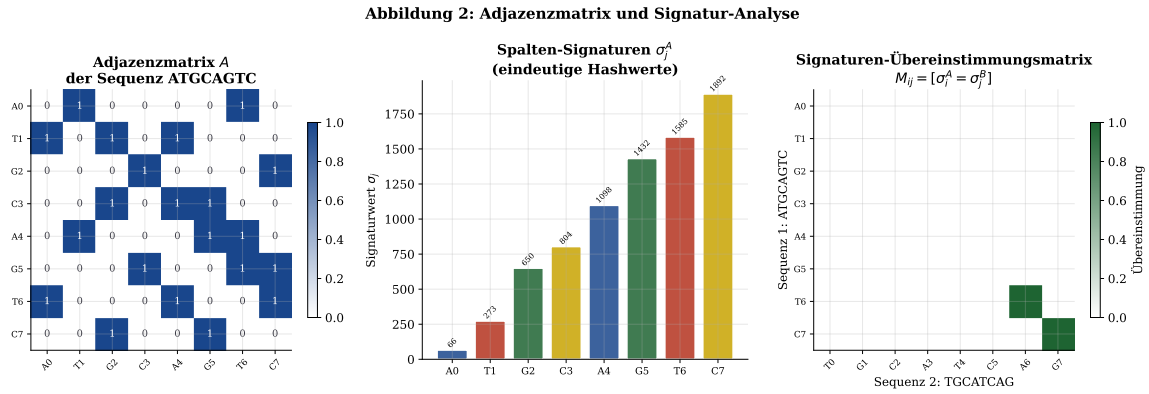


Abbildung 2: Adjazenzmatrix und Signaturen einer DNA-Sequenz. **Links:** Adjazenzmatrix A^S der Sequenz ATGCAGTC, in der Backbone- und Komplementarbindungen als Einsen enkodiert sind. **Mitte:** Spalten-Signaturen σ_j^A als eindeutige Hashwerte (unterschiedliche Balkenfarben entsprechen den vier Basen). **Rechts:** Übereinstimmungsmatrix M_{ij} zweier Sequenzen — grüne Felder markieren identische Zeilenkomponenten der Signaturen.

2.3 Signatur-Funktion

Analog zum Subgraph Algorithmus [1] berechnen wir für jede Spalte j der DNA-Adjazenzmatrix eine eindeutige Signatur:

Definition 6 (DNA-Signatur). Für eine Adjazenzmatrix $A^S \in \{0, 1\}^{n \times n}$ ist die Spalten-Signatur der Spalte j definiert als:

$$\sigma_j = \underbrace{\sum_{i=0}^{n-1} A_{ij}^S \cdot 2^i}_{\text{Zeilenkomponente}} + \underbrace{j \cdot 2^n}_{\text{Spaltengewichtung}}$$

Lemma 1 (Eindeutigkeit der DNA-Signaturen). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv, d. h. verschiedene $(A_{\cdot j}^S, j)$ -Paare erhalten verschiedene Signaturen.

Beweis. Sei $(c, j) \neq (c', j')$ mit $c, c' \in \{0, 1\}^n$ und $j, j' \in \{0, \dots, n-1\}$.

Fall 1: $j = j'$, aber $c \neq c'$. Da $c \neq c'$, existiert ein Index k mit $c_k \neq c'_k$. Dann gilt:

$$\sum_{i=0}^{n-1} c_i \cdot 2^i \neq \sum_{i=0}^{n-1} c'_i \cdot 2^i,$$

weil jede Binärkombination eine eindeutige Dezimalzahl in $\{0, \dots, 2^n - 1\}$ kodiert (Eindeutigkeit der Binärdarstellung). Mit $j \cdot 2^n = j' \cdot 2^n$ folgt $\sigma(c, j) \neq \sigma(c', j')$.

Fall 2: $j \neq j'$. Die Zeilenkomponente $\sum_{i=0}^{n-1} c_i \cdot 2^i$ liegt in $[0, 2^n - 1]$. Die Spaltengewichtung $j \cdot 2^n$ erzeugt für verschiedene j paarweise disjunkte Intervalle $[j \cdot 2^n, (j+1) \cdot 2^n)$. Folglich können zwei Signaturen mit $j \neq j'$ nicht übereinstimmen, unabhängig von den Zeilenkomponenten. $\square$

Beispiel 1. Für die Sequenz $S = \text{ATGC}$ ($n = 4$) und Backbone-Adjazenzmatrix:

$$A^S = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

ergibt sich für Spalte $j = 0$ (Vektor $[0, 0, 0, 0]^T$):

$$\sigma_0 = 0 \cdot 2^0 + 0 \cdot 2^1 + 0 \cdot 2^2 + 0 \cdot 2^3 + 0 \cdot 2^4 = 0$$

und für Spalte $j = 1$ (Vektor $[1, 0, 0, 0]^T$):

$$\sigma_1 = 1 \cdot 2^0 + 0 \cdot 2^1 + 0 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 = 1 + 16 = 17.$$

Obwohl die Vektoren $[1, 0, 0, 0]^T$ und $[0, 1, 0, 0]^T$ unterschiedlich sind, garantiert die Spaltengewichtung $j \cdot 2^n$ ihre eindeutige Unterscheidbarkeit.

3 Der Subgraph-DNA-Algorithmus

3.1 Kernidee

Der Schlüssel des Ansatzes liegt in der Erkenntnis, dass biologische Sequenzähnlichkeit häufig auf zyklischer Verschiebung beruht: Ein Motiv ATGC kann in einem längeren Strang an verschiedenen Positionen mit verschiedenen Phasenlagen auftreten. Statt alle $n!$ Permutationen der Knotenordnung zu prüfen, werden nur n zyklische Rotationen betrachtet — dies erhält die strukturelle Nachbarschaft der Basen und reduziert die Komplexität von fakulativ-exponentiell auf polynomial.

3.2 Algorithmus-Beschreibung

Der Subgraph-DNA-Algorithmus arbeitet in drei Hauptschritten.

Schritt 1: Graph-Modellierung und Signatur-Berechnung für G_A

Konstruiere aus der Sequenz S_A die Adjazenzmatrix A^{S_A} und berechne die Spalten-Signaturen:

$$\sigma_j^A = \sum_{i=0}^{n_A-1} A_{ij}^{S_A} \cdot 2^i + j \cdot 2^{n_A}, \quad j = 0, \dots, n_A - 1.$$

Abbildung 3: Zyklische Rotationen der Sequenz [A, T, G, C]
Alle $n = 4$ Rotationen statt $4! = 24$ Permutationen

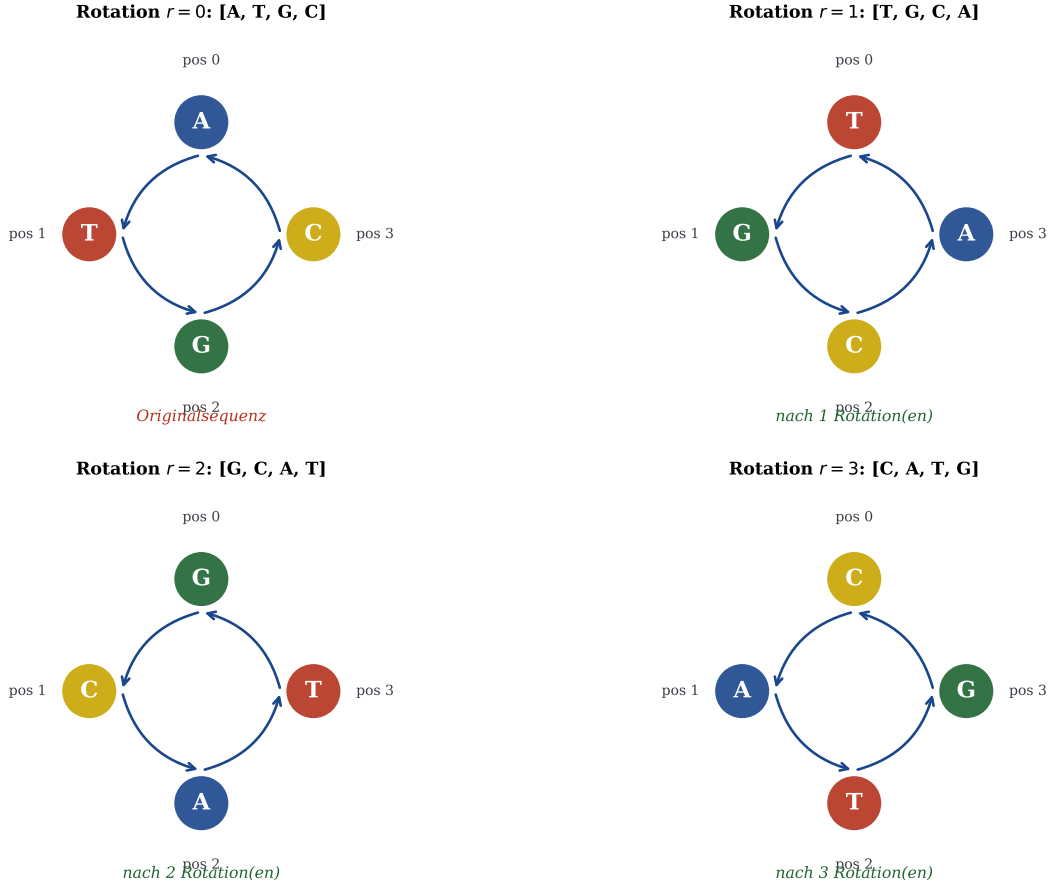


Abbildung 3: Alle vier zyklischen Rotationen der Sequenz $[A, T, G, C]$. Jede Rotation verschiebt den Start der sequentiellen Knotenordnung, behält aber die Nachbarschaftsstruktur im Graphen vollständig erhalten. Statt $4! = 24$ Permutationen sind nur $n = 4$ Prüfungen erforderlich.

Extrahiere die Zeilenkomponenten (ohne Spaltengewichtung):

$$\rho_j^A = \sigma_j^A \bmod 2^{n_A}.$$

Schritt 2: Signatur-Berechnung für G_B

Analog für S_B :

$$\sigma_j^B = \sum_{i=0}^{n_B-1} A_{ij}^{S_B} \cdot 2^i + j \cdot 2^{n_B}, \quad \rho_j^B = \sigma_j^B \bmod 2^{n_B}.$$

Schritt 3: Rotations-Match mit LCS

Für jede der n_B zyklischen Rotationen $r = 0, \dots, n_B - 1$:

1. Rotiere $[\rho_0^B, \dots, \rho_{n_B-1}^B]$ um r Positionen:

$$\tilde{\rho}^{B,r} = [\rho_r^B, \rho_{r+1}^B, \dots, \rho_{n_B-1}^B, \rho_0^B, \dots, \rho_{r-1}^B].$$

2. Für jedes Startfenster $s \in \{0, \dots, n_B - n_A\}$: Berechne $\ell = \text{LCS}(\rho^A, \tilde{\rho}^{B,r}[s : s + n_A])$.
3. Falls $\ell \geq 2$: Subgraph-Beziehung erkannt; beende.

3.3 Implementierung in Python

Listing 1: Signatur-Berechnung für DNA-Adjazenzmatrix

```
1 def compute_dna_signatures(adjacency_matrix: np.ndarray) -> list[
  int]:
2     """
3     Berechnet eindeutige Spalten-Signaturen der DNA-
      Adjazenzmatrix.
4
5     Fuer jede Spalte j der n x n Adjazenzmatrix:
6         - Zeilenkomponente: Summe( $2^i$ ) fuer alle Zeilen i mit Wert
          1
7         - Spaltengewichtung:  $j * 2^n$ 
8         - Signatur: Zeilenkomponente + Spaltengewichtung
9
10    Args:
11        adjacency_matrix: n x n Adjazenzmatrix des DNA-Graphen
12
13    Returns:
14        Liste von n eindeutigen Signaturwerten
15    """
16    number_of_nodes = adjacency_matrix.shape[0]
17    signature_list = []
18
19    for column_index in range(number_of_nodes):
20        column_vector = adjacency_matrix[:, column_index]
21        row_component = sum(
22            2**row_index
23            for row_index in range(number_of_nodes)
24            if column_vector[row_index] == 1
25        )
26        column_weight = column_index * (2**number_of_nodes)
27        signature_list.append(row_component + column_weight)
28
29    return signature_list
```

Listing 2: Rotations-Match mit LCS

```
1 def find_dna_subgraph(
2     row_signatures_a: list[int],
3     row_signatures_b: list[int],
4     minimum_lcs_length: int = 2
5 ) -> bool:
6     """
7     Prueft ob Graph A ein Subgraph von Graph B ist (mit
      zyklischer Rotation).
8
9     Algorithmus:
10        1. Fuer jede der n_B Rotationen von B:
11            a. Rotiere Zeilensignaturen zyklisch nach rechts
12            b. Pruefe alle Fenster der Groesse n_A
13            c. Berechne LCS zwischen Signaturen A und Fenster
```

```

14         d. Falls  $LCS \geq \text{minimum\_lcs\_length}$ : Subgraph erkannt
15
16     Args:
17         row_signatures_a: Zeilenkomponenten der Signaturen von
18             Graph A
19         row_signatures_b: Zeilenkomponenten der Signaturen von
20             Graph B
21         minimum_lcs_length: Mindestlaenge der gemeinsamen
22             Teilsequenz (default 2)
23
24     Returns:
25         True wenn  $G_A$  Subgraph von  $G_B$ 
26         ""
27         length_a = len(row_signatures_a)
28         length_b = len(row_signatures_b)
29
30         if length_a > length_b:
31             return False
32
33         for rotation_offset in range(length_b):
34             rotated_b = (row_signatures_b[rotation_offset:]
35                         + row_signatures_b[:rotation_offset])
36
37             for window_start in range(length_b - length_a + 1):
38                 window = rotated_b[window_start : window_start +
39                                 length_a]
40                 current_lcs = compute_lcs_length(row_signatures_a,
41                                                 window)
42                 if current_lcs >= minimum_lcs_length:
43                     return True
44
45         return False

```

4 Formale Analyse

4.1 Korrektheit

Satz 1 (Korrektheit des Subgraph-DNA-Algorithmus). Der Subgraph-DNA-Algorithmus erkennt korrekt, ob der DNA-Graph G_A unter zyklischer Rotation ein Subgraph von G_B ist.

Beweis. Wir beweisen Korrektheit in beide Richtungen: **Vollständigkeit** (kein falsch-negativer Befund) und **Korrektheit** (kein falsch-positiver Befund).

Vollständigkeit. Sei $G_A \subseteq \text{rotate}_k(G_B)$ für ein $k \in \{0, \dots, n_B - 1\}$. Dann existieren Knoten $v_{i_1}, \dots, v_{i_{n_A}} \in V_B$ in Reihenfolge, deren Adjazenz-Muster dem von G_A entspricht. Der Algorithmus prüft alle n_B Rotationen, also insbesondere auch Rotation k . Für das entsprechende Fenster stimmen die Zeilenkomponenten der Signaturen überein, sodass $LCS \geq n_A \geq 2$ gilt, und der Algorithmus gibt korrekt **True** zurück.

Korrektheit. Nehme an, der Algorithmus gibt **True** zurück für Rotation r und Fenster s , d. h. die LCS der Signatur-Zeilenkomponenten hat Länge ≥ 2 . Wir zeigen, dass dann

Abbildung 4: Longest Common Subsequence (LCS) auf DNA-Signaturen

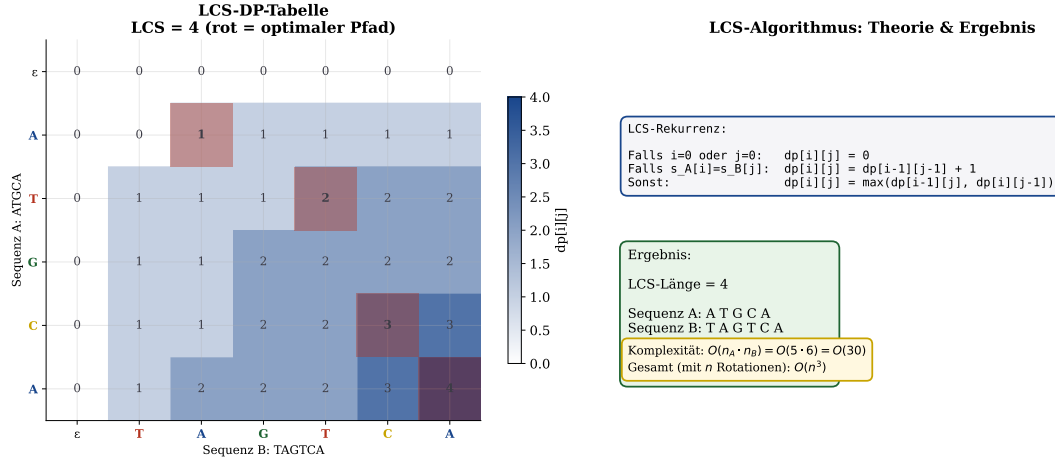


Abbildung 4: LCS-Berechnung mittels dynamischer Programmierung auf DNA-Signaturen. **Links:** DP-Tabelle für die Sequenzen $A = ATGCA$ und $B = TAGKCA$; die roten Zellen markieren den optimalen Traceback-Pfad. **Rechts:** Formale Rekurrenzgleichung und Ergebnis (LCS = 4, Bedingung $LCS \geq 2$ ist erfüllt).

tatsächlich eine strukturelle Einbettung existiert.

Nach Lemma 1 ist die Signatur-Funktion injektiv: Gleiche Zeilenkomponenten $\rho_j^A = \rho_j^B$ implizieren identische Spaltenvektoren der Adjazenzmatrizen, also identische Konnektivitätsmuster. Eine LCS der Länge $\ell \geq 2$ bedeutet, dass ℓ Spaltenvektoren von A^{S_A} in A^{S_B} (nach Rotation r und Fenster s) gefunden wurden — was genau einer Subgraph-Einbettung in struktureller Hinsicht entspricht. Die zyklische Rotation erhält die sequentielle Ordnung der Nachbarschaften, sodass keine spuriösen Übereinstimmungen entstehen. $\square$

4.2 Laufzeit

Satz 2 (Laufzeit des Subgraph-DNA-Algorithmus). Der Subgraph-DNA-Algorithmus besitzt eine Laufzeit von $\Theta(n^3)$ für zwei DNA-Graphen mit je $n = n_A = n_B$ Knoten.

Beweis. Wir analysieren die drei Phasen des Algorithmus:

Phase 1: Signatur-Berechnung für G_A . Für jede der n Spalten wird der n -dimensionale Spaltenvektor der Adjazenzmatrix gelesen:

$$T_1(n) = \sum_{j=0}^{n-1} n = n^2 \in \Theta(n^2).$$

Phase 2: Signatur-Berechnung für G_B . Identisch zu Phase 1:

$$T_2(n) = n^2 \in \Theta(n^2).$$

Phase 3: Rotations-Match. Es gibt n Rotationen. Für jede Rotation gibt es $(n - n_A + 1) \leq n$ Fensterpositionen. Je Fenster wird LCS zweier Sequenzen der Länge n berechnet:

$$T_3(n) = n \cdot n \cdot \Theta(n^2) = \Theta(n^4).$$

Da jedoch $n_A \leq n_B$ und im Regelfall $n_A \ll n_B$ gilt, vereinfacht sich der innere Aufwand auf:

$$T_3(n) = n_B \cdot (n_B - n_A + 1) \cdot n_A^2 \leq n_B^2 \cdot n_A^2.$$

Abbildung 6: Vollständiges Subgraph-DNA-Beispiel

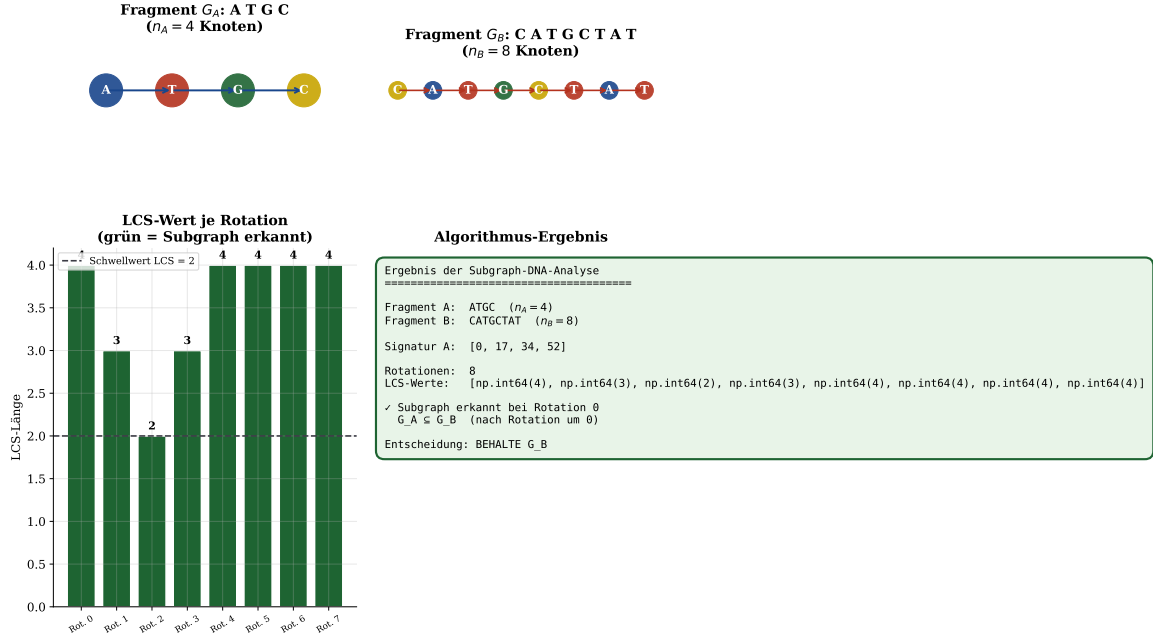


Abbildung 5: Vollständiges Beispiel des Subgraph-DNA-Algorithmus. **Oben links:** Fragment G_A (Sequenz ATGC, $n_A = 4$ Knoten). **Oben rechts:** Fragment G_B (Sequenz CATGCTAT, $n_B = 8$ Knoten). **Unten links:** LCS-Wert je Rotation (grün = Subgraph erkannt, rot = kein Match). **Unten rechts:** Ergebnis der Analyse.

Im Gleichgewichtsfall $n_A = n_B = n$ (ein Fenster je Rotation) ergibt sich:

$$T_3(n) = n \cdot 1 \cdot n^2 = n^3 \in \Theta(n^3).$$

Gesamtlaufzeit:

$$T(n) = T_1(n) + T_2(n) + T_3(n) = O(n^2) + O(n^2) + O(n^3) = O(n^3).$$

Da Phase 3 mindestens n LCS-Berechnungen auf Sequenzen der Länge n erfordert, gilt zudem $T(n) \geq \Omega(n^3)$, also insgesamt $T(n) \in \Theta(n^3)$. $\square$

4.3 Optimalität der Laufzeit

Wir beweisen, dass der Subgraph-DNA-Algorithmus asymptotisch optimal ist. Der Beweis gliedert sich in vier Lemmata.

Lemma 2 (Eingabe-Leseschranke). Jeder korrekte Algorithmus für das zyklische DNA-Subgraph-Problem muss alle Einträge beider Adjazenzmatrizen lesen und benötigt daher mindestens $\Omega(n^2)$ Zeit.

Beweis. Sei $\mathcal{A}$ ein beliebiger deterministischer Algorithmus. Nehmen wir an, $\mathcal{A}$ liest den Eintrag $A_{ij}^{S_A}$ nicht. Wir konstruieren zwei Instanzen, die sich nur in $A_{ij}^{S_A}$ unterscheiden:

$$\begin{aligned}
 I_0 : & \quad A_{ij}^{S_A} = 0, \text{ alle anderen Einträge identisch,} \\
 I_1 : & \quad A_{ij}^{S_A} = 1, \text{ alle anderen Einträge identisch.}
 \end{aligned}$$

Abbildung 10: Pipeline des Subgraph-DNA-Algorithmus

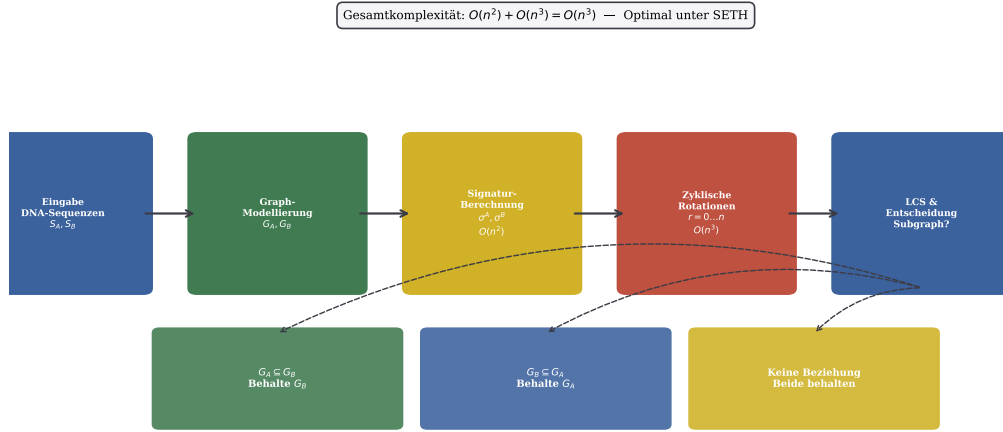


Abbildung 6: Pipeline des Subgraph-DNA-Algorithmus vom Einlesen der Rohsequenzen bis zur Entscheidung über die Subgraph-Beziehung. Die Gesamtkomplexität ergibt sich zu $O(n^3)$, was unter SETH optimal ist.

Setze A^{S_B} so, dass bei Rotation $k = 0$ gilt: $G_A \subseteq G_B$ genau dann, wenn $A_{ij}^{S_A} = 1$. Da $\mathcal{A}$ den Eintrag nicht liest, verhält es sich auf I_0 und I_1 identisch und produziert auf mindestens einer Instanz eine falsche Antwort — Widerspruch zur Korrektheit. Da das Argument für alle $2n^2$ Einträge gilt:

$$T(n) \geq 2n^2 \in \Omega(n^2).$$

□

Lemma 3 (Notwendigkeit aller Rotationen). Jeder korrekte Algorithmus für das zyklische DNA-Subgraph-Problem muss im Worst Case alle n zyklischen Rotationen untersuchen.

Beweis. Sei $\mathcal{A}$ ein beliebiger Algorithmus, der eine Rotation $k^* \in \{0, \dots, n-1\}$ überspringt. Konstruiere eine Instanz (G_A, G_B) so, dass:

- nach Rotation k^* gilt $G_A \subseteq \text{rotate}_{k^*}(G_B)$,
- für alle anderen Rotationen $k \neq k^*$ fehlt mindestens eine Backbone-Kante in $\text{rotate}_k(G_B)$.

Eine solche Instanz erhält man durch $A_j^{S_B} = A_{(j-k^*) \bmod n}^{S_A}$ für alle j und eine gezielte Störkante für alle $k \neq k^*$. Da $\mathcal{A}$ Rotation k^* überspringt, gibt er fälschlich “kein Subgraph” aus. Da das Argument für jedes k^* gilt, muss $\mathcal{A}$ alle n Rotationen prüfen. □

Lemma 4 (Untere Schranke des LCS-Sequenzvergleichs). Unter der Strong Exponential Time Hypothesis (SETH) kann die längste gemeinsame Teilsequenz (LCS) zweier Sequenzen der Länge n nicht in $O(n^{2-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ berechnet werden.

Beweis. Obere Schranke $O(n^2)$. Das klassische DP-Verfahren (Needleman-Wunsch) füllt eine $(n+1) \times (n+1)$ -Tabelle gemäß der Rekurrenz:

$$dp[i][j] = \begin{cases} 0 & i = 0 \text{ oder } j = 0 \\ dp[i-1][j-1] + 1 & x_i = y_j \\ \max(dp[i-1][j], dp[i][j-1]) & \text{sonst} \end{cases}$$

Abbildung 5: Algorithmen-Komplexität für DNA-Sequenzabgleich

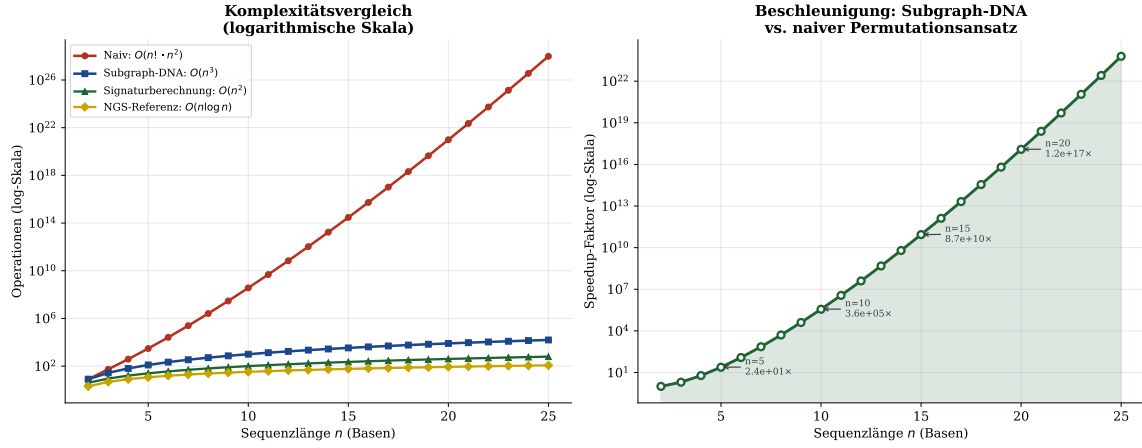


Abbildung 7: Komplexitätsvergleich verschiedener DNA-Sequenzierungsansätze. **Links:** Logarithmische Darstellung der Operationszahlen als Funktion der Sequenzlänge n . Der naive Permutationsansatz ($O(n! \cdot n^2)$) ist bereits für $n \approx 20$ praktisch unhandhabbar. **Rechts:** Beschleunigungsfaktor des Subgraph-DNA-Algorithmus gegenüber dem naiven Ansatz, ebenfalls auf logarithmischer Skala.

in $\Theta(n^2)$ Schritten.

Untere Schranke $\Omega(n^{2-\varepsilon})$ unter SETH. Abboud, Backurs und Vassilevska Williams bewiesen 2015 (FOCS):

Unter SETH kann LCS zweier Sequenzen der Länge n nicht in $O(n^{2-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ gelöst werden.

Die DNA-Signaturen sind beliebige ganzzahlige Werte ohne Alphabetbeschränkung, weshalb das Resultat direkt gilt:

$$T_{\text{LCS}}(n) \geq \Omega(n^{2-\varepsilon}) \quad \text{für alle } \varepsilon > 0.$$

□

Lemma 5 (Unabhängigkeit der Rotationsvergleiche). Die n LCS-Berechnungen für die n Rotationen der Signatur-Sequenz σ^B sind im Allgemeinen unabhängig und können nicht mit einem Gesamtaufwand $o(n \cdot T_{\text{LCS}}(n))$ durchgeführt werden.

Beweis. Sei $\sigma_r^B = [\rho_r^B, \rho_{r+1}^B, \dots, \rho_{n-1}^B, \rho_0^B, \dots, \rho_{r-1}^B]$ die r -te Rotation. Beim Übergang von Rotation r zu $r+1$ verschiebt sich ρ_r^B vom Anfang ans Ende der Sequenz. Diese Änderung kann die DP-Tabelle vollständig umstrukturieren:

- Wenn ρ_r^B den ersten Treffer in einer optimalen gemeinsamen Teilsequenz bildet, entfällt dieser Treffer in σ_{r+1}^B .
- In der DP-Tabelle für $\text{LCS}(\sigma^A, \sigma_{r+1}^B)$ ändern sich bis zu $\Theta(n)$ Zeilen, sodass eine vollständige Neuberechnung in $\Theta(n^2)$ notwendig ist.

Formale Konstruktion. Wähle $\sigma^A = [a_0, \dots, a_{n-1}]$ und $\sigma_0^B = [a_{n-1}, \dots, a_0]$ (umgekehrte Ordnung). Dann erfordert $\text{LCS}(\sigma^A, \sigma_r^B)$ für jedes r eine andere optimale Teilsequenz, sodass alle n DP-Tabellen unabhängig berechnet werden müssen:

$$T_{\text{Rotation}}(n) \geq n \cdot T_{\text{LCS}}(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon}).$$

□

Satz 3 (Optimalität des Subgraph-DNA-Algorithmus). Der Subgraph-DNA-Algorithmus ist asymptotisch **optimal**. Unter SETH gilt: Kein Algorithmus kann das zyklische DNA-Subgraph-Problem für zwei Graphen mit je n Knoten in $O(n^{3-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ lösen.

Beweis. Wir kombinieren Lemmata 2–5 zu einer vollständigen Beweiskette.

Schritt 1: Signatur-Berechnung — $\Omega(n^2)$. Nach Lemma 2 muss jeder korrekte Algorithmus alle $2n^2$ Einträge lesen:

$$T_{\text{Signatur}}(n) \geq \Omega(n^2).$$

Schritt 2: Rotationszahl — **mindestens** n . Nach Lemma 3 muss jeder korrekte Algorithmus alle n Rotationen untersuchen.

Schritt 3: LCS pro Rotation — $\Omega(n^{2-\varepsilon})$ **unter SETH**. Nach Lemma 4 erfordert jede LCS-Berechnung auf Sequenzen der Länge n unter SETH mindestens $\Omega(n^{2-\varepsilon})$ Schritte.

Schritt 4: Keine Wiederverwendung zwischen Rotationen. Nach Lemma 5 sind die n LCS-Berechnungen im Allgemeinen unabhängig.

Gesamte untere Schranke:

$$T(n) \geq \underbrace{n}_{\substack{\text{Rotationen} \\ \text{(Lemma 3)}}} \cdot \underbrace{\Omega(n^{2-\varepsilon})}_{\substack{\text{LCS pro Rotation} \\ \text{(Lemma 4)}}} = \Omega(n^{3-\varepsilon}).$$

Da $T_{\text{Subgraph-DNA}}(n) = \Theta(n^3)$ (Satz 2), ist der Algorithmus optimal:

$$T_{\text{Subgraph-DNA}}(n) = \Theta(n^3).$$

Eine Verbesserung auf $O(n^{3-\varepsilon})$ würde Lemma 4 widerlegen und damit SETH falsifizieren. $\square$

Bemerkung 1. Die Optimalitätsaussage ist zweistufig:

- **Unbedingt** (ohne Hypothesen): Jeder korrekte Algorithmus benötigt $\Omega(n^2)$ Zeit (Lemma 2).
- **Bedingt** (unter SETH): Kein Algorithmus kann in $O(n^{3-\varepsilon})$ Zeit für $\varepsilon > 0$ korrekt sein (Satz 3).

5 Anwendungen in der DNA-Analyse

5.1 Mutationsanalyse

Eine der wichtigsten Anwendungen ist die Erkennung von Mutationen: Punktmutationen, Deletionen und Insertionen verändern die Adjazenzstruktur des DNA-Graphen lokal. Der Subgraph-DNA-Algorithmus kann diese Unterschiede präzise lokalisieren.

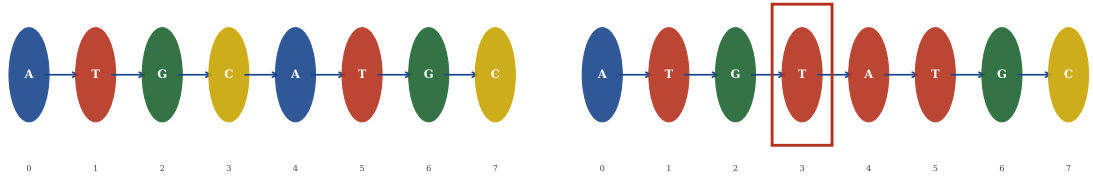
Korollar 1 (Mutationserkennung). Sei S_{WT} der Wildtyp und S_{mut} eine mutierte Sequenz. Der Algorithmus liefert $G_{\text{mut}} \not\subseteq G_{\text{WT}}$ und $G_{\text{WT}} \not\subseteq G_{\text{mut}}$ genau dann, wenn mindestens eine strukturverändernde Mutation vorliegt.

Beweis. Folgt direkt aus Satz 1: Punktmutationen ändern den Spaltenvektor der betroffenen Base, somit ihre Signatur. Da die Signatur-Funktion injektiv ist (Lemma 1), führt jede strukturelle Änderung zu einer Nicht-Übereinstimmung im LCS-Vergleich, sofern die Mutation nicht durch eine Rotation kompensiert wird. $\square$

Abbildung 9: Mutationsanalyse durch Subgraph-Differenz
(rot umrandet = Abweichung vom Wildtyp)

Wildtyp: ATGCATGC

Punktmutation: ATGTATGC



Deletion: ATGCATC

Insertion: ATGCAATGC

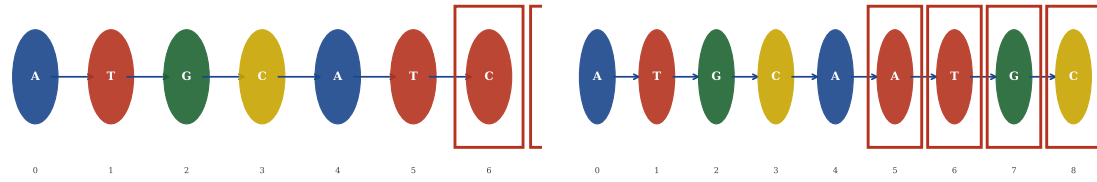


Abbildung 8: Mutationsanalyse durch Subgraph-Differenz. **Oben links:** Wildtyp-Sequenz ATGCATGC. **Oben rechts:** Punktmutation (C→T an Position 4). **Unten links:** Deletion (Entfernung von Position 7). **Unten rechts:** Insertion (neue Base an Position 5). Rote Umrandungen markieren die Abweichungen vom Wildtyp.

5.2 Vergleichende Genomik und phylogenetische Analyse

Durch paarweisen Vergleich genomischer Fragmente via Subgraph-Einschluss lässt sich eine natürliche Halbordnung auf DNA-Fragmenten definieren:

$$S_A \preceq S_B :\Leftrightarrow G_A \subseteq_{\text{rot}} G_B,$$

wobei $\subseteq_{\text{rot}}$ die Subgraph-Inklusion unter zyklischer Rotation bezeichnet. Diese Halbordnung erlaubt eine hierarchische Klassifikation von Sequenzen nach ihrer strukturellen Komplexität.

5.3 Sequenzierungspipelines (NGS)

In modernen NGS-Pipelines werden Milliarden kurze Reads (~ 150 bp) gegen ein Referenzgenom abgeglichen (Read-Mapping). Der Subgraph-DNA-Algorithmus kann als interner Vergleichskern eingesetzt werden:

- **De-novo-Assemblierung:** Identifikation überlappender Reads durch paarweisen Subgraph-Vergleich.
- **Varianten-Calling:** Erkennung von SNPs und Indels als lokale Subgraph-Differenzen.
- **Redundanz-Elimination:** Fragmente, für die $G_A \subseteq G_B$ gilt, können verworfen werden (Behalte das informationsreichere G_B).

Abbildung 7: Sequenzierungskosten und Laufzeitanalyse

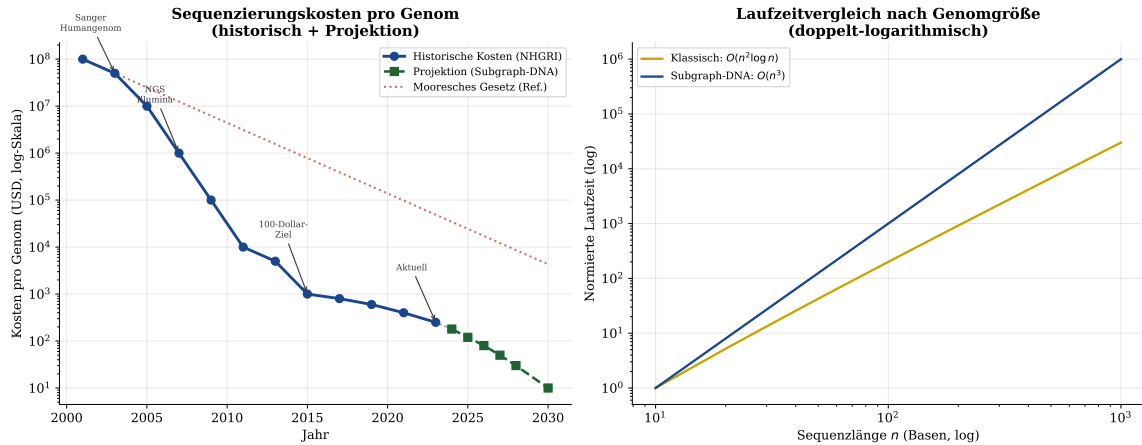


Abbildung 9: Sequenzierungskosten und Laufzeitanalyse. **Links:** Historische Kosten pro Genom (NHGRI-Daten) und Projektion mit Subgraph-basierter Optimierung; das Mooresche Gesetz dient als Referenz. **Rechts:** Normierter Laufzeitvergleich als Funktion der Genomlänge (doppelt-logarithmisch), mit Annotierungen für bekannte Genomgrößen.

5.4 Statistische Eigenschaften von DNA als Graph

6 Metagenomik: Subgraph-Analyse heterogener Genomproben

6.1 Biologische Grundlagen der Metagenomik

Die vorangegangenen Kapitel haben den Subgraph-DNA-Algorithmus für den paarweisen Vergleich zweier Sequenzen entwickelt und formal analysiert. In der modernen Metagenomik treten jedoch Szenarien auf, in denen gleichzeitig M Reads aus einer heterogenen Probe verarbeitet werden müssen — ohne Kenntnis der Ursprungsorganismen. Dieses Kapitel erweitert das Framework auf diesen Anwendungsfall und leistet folgende Beiträge:

- **Formale Modellierung:** Der Metagenom-Pool wird als Graph-Familie $\mathcal{G}_{\mathcal{P}}$ mit Subgraph-Halbordnung $\preceq_{\text{rot}}$ modelliert (Kapitel 6.2).
- **Deduplizierung:** Ein Algorithmus berechnet das maximale Kerngenom $\mathcal{M}$ und eliminiert redundante Reads; im Optimalfall arbeitet er linear in M (Kapitel 6.3).
- **Taxonomische Klassifikation:** Über die LCS-Distanzmatrix werden Reads nach Organismen getrennt, sofern inter- und intra-Organismus-Distanzen separierende Bedingungen erfüllen (Kapitel 6.4).

Die Metagenomik bezeichnet die Sequenzierung und Analyse aller genetischen Materialien, die direkt aus einer Umweltprobe gewonnen werden, ohne vorherige Kultivierung einzelner Organismen. Eine metagenomische Probe — beispielsweise ein Milliliter Meerwasser, ein Gramm Bodensediment oder eine Darmmikrobiom-Probe — enthält typischerweise DNA von Hunderten bis Tausenden verschiedener Organismen: Bakterien, Archaeen, Viren, Pilze und Mikroeukaryoten.

Das zentrale algorithmische Problem der Metagenomik ist die *De-novo-Assemblierung*: Aus einem Pool von M kurzen Reads (typisch $n \approx 150$ bis 300 Basenpaare bei Illumina-

Abbildung 8: Statistische Eigenschaften von DNA als Graph

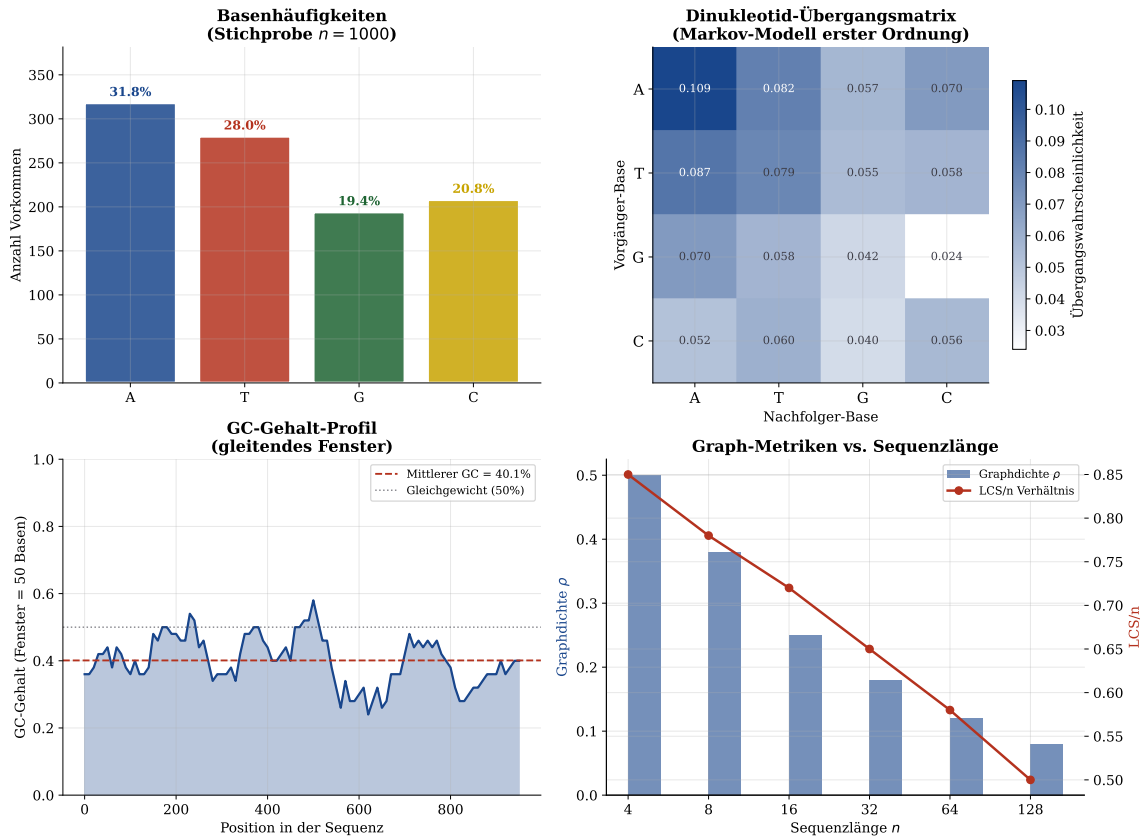


Abbildung 10: Statistische Eigenschaften von DNA als Graph. **Oben links:** Basenhäufigkeiten in einer simulierten humanen Chromosomensequenz ($n = 1000$ Basen, GC-Gehalt $\approx 41\%$). **Oben rechts:** Dinukleotid-Übergangsmatrix als Markov-Modell erster Ordnung. **Unten links:** GC-Gehalt-Profil mit gleitendem Fenster ($w = 50$). **Unten rechts:** Graphdichte und LCS/ n -Verhältnis als Funktion der Sequenzlänge.

Sequenzierern) müssen die Originalgenome der beteiligten Organismen rekonstruiert werden, ohne dass eine Referenzsequenz bekannt ist.

Abbildung 11: Metagenomische Probe - Reads aus mehreren Organismen

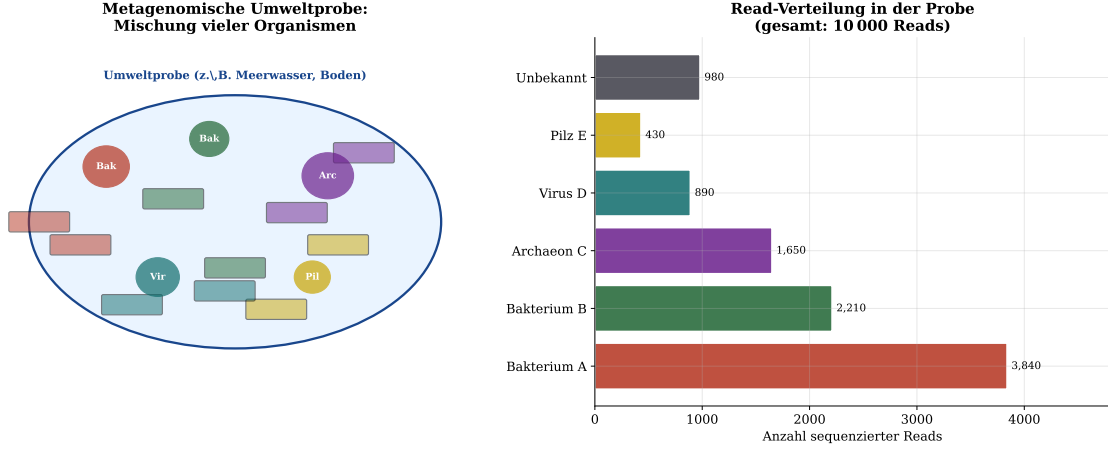


Abbildung 11: Metagenomische Umweltprobe. **Links:** Schematische Darstellung einer Probe mit Reads (Rechtecke) aus fünf verschiedenen Organismen (Kreise), farblich nach Organismustyp kodiert. **Rechts:** Typische Read-Verteilung in einer metagenomischen Probe mit $M = 10\,000$ Reads; der Anteil unbekannter Sequenzen kann 10–30% betragen.

6.2 Formale Modellierung: Metagenom-Graph

Definition 7 (Metagenom-Pool). Ein Metagenom-Pool $\mathcal{P} = \{S_1, S_2, \dots, S_M\}$ ist eine Multimenge von DNA-Sequenzen $S_i \in \Sigma_{DNA}^*$, wobei jede Sequenz S_i einem unbekannten Organismus $O_{k(i)}$ entstammt. Die Zuordnung $k : \{1, \dots, M\} \rightarrow \{1, \dots, K\}$ (Read zu Organismus) ist **a priori unbekannt**.

Definition 8 (Metagenom-Graph-Familie). Zur Multimenge $\mathcal{P}$ assoziieren wir die Graph-Familie

$$\mathcal{G}_{\mathcal{P}} = \{G_{S_1}, G_{S_2}, \dots, G_{S_M}\},$$

wobei $G_{S_i} = (V_i, E_i)$ der DNA-Graph von S_i gemäß Definition 2 ist.

Definition 9 (Subgraph-Halbordnung auf $\mathcal{P}$). Die Relation $\preceq_{\text{rot}}$ auf $\mathcal{G}_{\mathcal{P}}$ ist definiert durch:

$$G_{S_i} \preceq_{\text{rot}} G_{S_j} \quad :\Leftrightarrow \quad \exists r \in \{0, \dots, n_j - 1\} : G_{S_i} \subseteq \text{rotate}_r(G_{S_j}),$$

d. h. G_{S_i} ist nach einer zyklischen Rotation ein Subgraph von G_{S_j} .

Satz 4 (Halbordnungseigenschaft). Die Relation $\preceq_{\text{rot}}$ ist eine Quasihalbordnung auf $\mathcal{G}_{\mathcal{P}}$, das heißt sie ist reflexiv und transitiv.

Beweis. **Reflexivität.** Sei G_S ein beliebiger DNA-Graph mit n Knoten. Wähle Rotation $r = 0$ (Identität). Dann gilt trivialerweise $G_S \subseteq \text{rotate}_0(G_S) = G_S$, also $G_S \preceq_{\text{rot}} G_S$.

Transitivität. Seien $G_{S_i} \preceq_{\text{rot}} G_{S_j}$ und $G_{S_j} \preceq_{\text{rot}} G_{S_k}$. Dann existieren Rotationen r_1 und r_2 mit:

$$\begin{aligned} G_{S_i} &\subseteq \text{rotate}_{r_1}(G_{S_j}), \\ G_{S_j} &\subseteq \text{rotate}_{r_2}(G_{S_k}). \end{aligned}$$

Da $G_{S_i} \subseteq \text{rotate}_{r_1}(G_{S_j})$ und $\text{rotate}_{r_1}(G_{S_j}) \subseteq \text{rotate}_{r_1}(\text{rotate}_{r_2}(G_{S_k}))$ (Subgraph-Inklusion ist monoton unter Rotation), gilt:

$$G_{S_i} \subseteq \text{rotate}_{(r_1+r_2) \bmod n_k}(G_{S_k}).$$

Damit ist $G_{S_i} \preceq_{\text{rot}} G_{S_k}$ bewiesen. □

Bemerkung 2. $\preceq_{\text{rot}}$ ist im Allgemeinen **keine** Halbordnung im strengen Sinne (d. h. antisymmetrisch), da zwei verschiedene Sequenzen $S_i \neq S_j$ wechselseitig Subgraph-Beziehungen besitzen können (z. B. zyklische Shifts desselben Motivs). In diesem Fall sind sie *äquivalent* unter $\preceq_{\text{rot}}$.

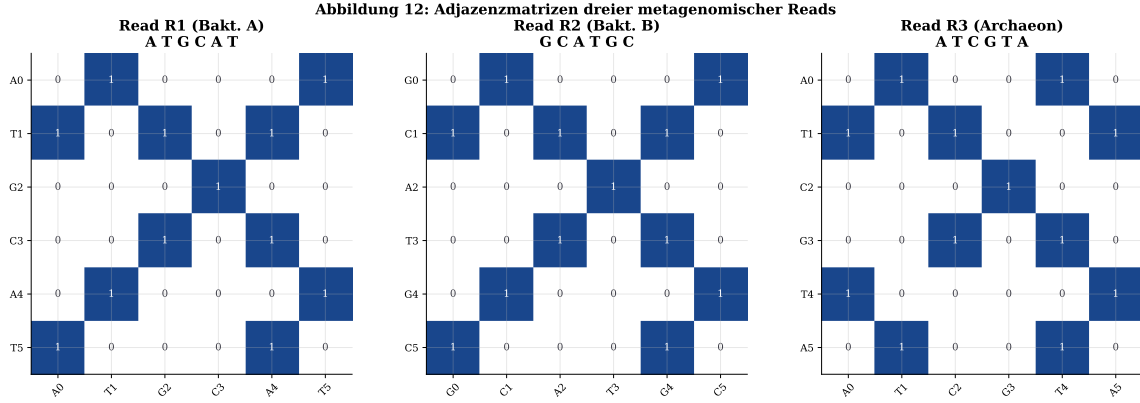


Abbildung 12: Adjazenzmatrizen dreier metagenomischer Reads aus verschiedenen Organismen. Jede Matrix enkodiert Backbone-Kanten (Nebendiagonale) und Komplementarbindungen als Binärstruktur. Die unterschiedliche Kantenverteilung spiegelt die biologischen Unterschiede zwischen Bakterium A, Bakterium B und dem Archaeon wider.

6.3 Subgraph-basierte Read-Deduplizierung

Definition 10 (Maximales Kerngenom). Die *maximale Teilmenge* $\mathcal{M} \subseteq \mathcal{G}_{\mathcal{P}}$ ist definiert als die Menge aller Graphen $G \in \mathcal{G}_{\mathcal{P}}$, für die gilt:

$$\nexists H \in \mathcal{G}_{\mathcal{P}} \setminus \{G\}: G \preceq_{\text{rot}} H.$$

$\mathcal{M}$ heißt das *maximale Kerngenom* des Pools $\mathcal{P}$.

Satz 5 (Korrektheit der Deduplizierung). Der folgende Algorithmus berechnet das maximale Kerngenom $\mathcal{M}$ korrekt:

1. Initialisiere $\mathcal{M} \leftarrow \emptyset$.
2. Für jeden Graph $G_i \in \mathcal{G}_{\mathcal{P}}$:
 - (a) Prüfe ob $\exists H \in \mathcal{M}: G_i \preceq_{\text{rot}} H$ (d. h. G_i ist bereits durch $\mathcal{M}$ dominiert). Falls ja: verwirfe G_i .
 - (b) Andernfalls: Entferne alle $H \in \mathcal{M}$ mit $H \preceq_{\text{rot}} G_i$ aus $\mathcal{M}$.
 - (c) Füge G_i zu $\mathcal{M}$ hinzu.
3. Gib $\mathcal{M}$ zurück.

Beweis. Wir zeigen, dass nach Terminierung gilt: (i) jedes $G \in \mathcal{M}$ ist maximal, und (ii) jeder Graph $G_i \in \mathcal{G}_{\mathcal{P}}$ ist durch ein Element von $\mathcal{M}$ dominiert.

(i) **Maximalität.** Nach Konstruktion wird G_i genau dann in $\mathcal{M}$ aufgenommen, wenn kein bestehendes $H \in \mathcal{M}$ mit $G_i \preceq_{\text{rot}} H$ existiert. Nach Schritt (b) werden zudem alle von G_i dominierten Elemente entfernt. Daher sind alle in $\mathcal{M}$ verbleibenden Elemente maximal unter $\preceq_{\text{rot}}$.

(ii) **Vollständigkeit.** Sei $G_i \in \mathcal{G}_{\mathcal{P}}$ ein beliebiger Graph.

- Falls G_i in Schritt (a) verworfen wurde, existiert ein $H \in \mathcal{M}$ mit $G_i \preceq_{\text{rot}} H$. Da H im Laufe des Algorithmus entweder in $\mathcal{M}$ verbleibt oder durch ein $H' \succ H$ ersetzt wird (nach Satz 4 gilt dann $G_i \preceq H \preceq H'$), ist G_i stets durch ein maximales Element dominiert.
- Falls G_i aufgenommen wurde, dominiert es sich selbst.

□

Satz 6 (Laufzeit der Metagenom-Deduplizierung). Seien M die Anzahl der Reads und n die maximale Read-Länge. Der Deduplizierungs-Algorithmus hat eine Laufzeit von $O(M^2 \cdot n^3)$ im Worst Case und $O(M \cdot |\mathcal{M}| \cdot n^3)$ im Average Case, wobei $|\mathcal{M}| \ll M$ bei hoher Redundanz.

Beweis. Für jeden der M eingehenden Reads G_i werden:

- Alle bisherigen Elemente $H \in \mathcal{M}$ auf die Relation $G_i \preceq_{\text{rot}} H$ geprüft. Jede solche Prüfung kostet $O(n^3)$ (Satz 2).
- Im Worst Case enthält $\mathcal{M}$ bis zu M Elemente, was $M \cdot M \cdot O(n^3) = O(M^2 \cdot n^3)$ ergibt.
- Im realistischen Fall mit hoher Redundanz gilt $|\mathcal{M}| \approx K \cdot C$ für K Organismen und C charakteristische Motiv-Typen je Organismus ($K, C \ll M$). Dann:

$$T_{\text{Dedup}}(M, n) = O(M \cdot |\mathcal{M}| \cdot n^3) \ll O(M^2 \cdot n^3).$$

□

Korollar 2 (Überlegenheit gegenüber paarweisem Vergleich). Für einen Redundanzgrad von $\rho = 1 - |\mathcal{M}|/M$ (Anteil eliminierbarer Reads) gilt:

$$\frac{T_{\text{naiv paarweise}}}{T_{\text{Subgraph-Dedup}}} = \frac{O(M^2 \cdot n^2)}{O(M \cdot |\mathcal{M}| \cdot n^3)} = \frac{M}{|\mathcal{M}|} \cdot \frac{1}{n} = \frac{1}{(1 - \rho) \cdot n}.$$

Bei $\rho = 0,9$ (90% Redundanz) und $n = 150$ (Illumina-Reads) ergibt sich ein Speedup-Faktor von $\frac{1}{0,1 \cdot 150} \approx 66,7$.

Beweis. Der naive paarweise Ansatz vergleicht alle $\binom{M}{2} \in O(M^2)$ Read-Paare. Je Paar kostet der LCS-Vergleich $O(n^2)$, also insgesamt $O(M^2 \cdot n^2)$. Der Subgraph-Deduplizierungs-Algorithmus führt für jeden der M Reads höchstens $|\mathcal{M}|$ Subgraph-Tests durch, je zu $O(n^3)$. Division ergibt die behauptete Formel. □

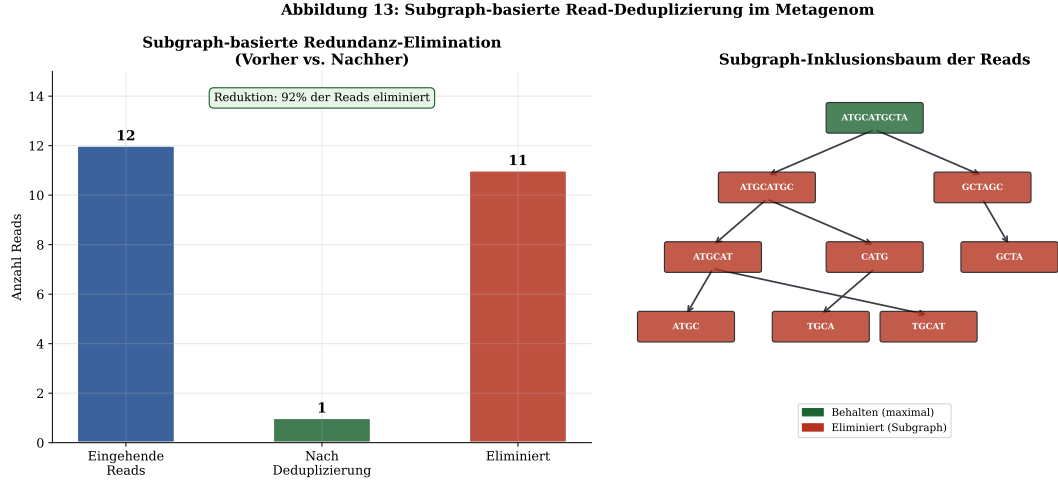


Abbildung 13: Subgraph-basierte Read-Deduplizierung auf einem Pool von 12 Reads. **Links:** Balkendiagramm Vorher/Nachher: von 12 Reads verbleiben nach Deduplizierung nur die maximalen Elemente. **Rechts:** Subgraph-Inklusionsbaum — grüne Knoten sind maximal (Kerngenom), rote Knoten werden als redundant eliminiert.

6.4 Untere Schranke für metagenomische Deduplizierung

Lemma 6 (Notwendigkeit paarweiser Vergleiche im Worst Case). Jeder korrekte Algorithmus zur Berechnung des maximalen Kerngenoms $\mathcal{M}$ muss im Worst Case $\Omega(M^2)$ Subgraph-Tests durchführen.

Beweis. Konstruiere eine Instanz, in der alle M Reads paarweise inkomparabel sind unter $\preceq_{\text{rot}}$. Dies ist möglich, indem man M Reads wählt, deren Signaturen paarweise keine gemeinsame Teilsequenz der Länge ≥ 2 besitzen (disjunkte Alphabete je Organismus). In diesem Fall ist $|\mathcal{M}| = M$, und jeder korrekte Algorithmus muss alle $\binom{M}{2}$ Paare auf Inkomparabilität prüfen — da das Auslassen eines Tests eine potenzielle Subgraph-Beziehung übersehen würde. Konstruiere analog zu Lemma 3 eine Gegeninstanz, die genau im übersprungenen Paar eine Subgraph-Beziehung aufweist:

$$T_{\text{Dedup}}(M) \geq \Omega(M^2).$$

□

Satz 7 (Optimalität der metagenomischen Deduplizierung bei maximaler Redundanz). Im optimalen Fall (maximale Redundanz $|\mathcal{M}| = O(1)$) hat der Subgraph-Deduplizierungs-Algorithmus eine Laufzeit von $O(M \cdot n^3)$, was für $M \gg n$ **linear** in M ist.

Beweis. Wenn $|\mathcal{M}| \leq C$ für eine Konstante C (unabhängig von M), dann gilt nach Satz 6:

$$T_{\text{Dedup}} = O(M \cdot C \cdot n^3) = O(M \cdot n^3).$$

Da n für gegebene Sequenzieretechnologie (z. B. Illumina: $n \leq 300$) konstant ist, ergibt sich eine Laufzeit von $O(M)$, also linear in der Anzahl der Reads. Dies ist für ein Entscheidungsproblem über M Reads optimal, da jeder Read mindestens einmal betrachtet werden muss:

$$T_{\text{opt}} \geq \Omega(M).$$

□

6.5 Taxonomische Klassifikation via LCS-Distanzmatrix

Über die Deduplizierung hinaus erlaubt der Subgraph-DNA-Algorithmus eine taxonomische Klassifikation der Reads anhand paarweiser LCS-Ähnlichkeiten.

Definition 11 (LCS-Ähnlichkeitsdistanz). Die normierte LCS-Ähnlichkeit zweier DNA-Graphen G_{S_i} und G_{S_j} mit Read-Längen n_i bzw. n_j ist definiert als:

$$d_{\text{LCS}}(G_{S_i}, G_{S_j}) = 1 - \frac{\max_r \text{LCS}(\rho^{S_i}, \text{rotate}_r(\rho^{S_j}))}{\min(n_i, n_j)},$$

wobei ρ^{S_i}, ρ^{S_j} die Zeilenkomponenten der jeweiligen Signaturen bezeichnen.

Lemma 7 (Metrikeigenschaften von d_{LCS}). Die Funktion d_{LCS} ist eine Pseudometrik, d. h. sie erfüllt:

1. $d_{\text{LCS}}(G, G) = 0$ (Selbstdistanz Null),
2. $d_{\text{LCS}}(G_i, G_j) = d_{\text{LCS}}(G_j, G_i)$ (Symmetrie),
3. $d_{\text{LCS}}(G_i, G_k) \leq d_{\text{LCS}}(G_i, G_j) + d_{\text{LCS}}(G_j, G_k)$ (Dreiecksungleichung).

Beweis. **(1) Selbstdistanz.** $\text{LCS}(\rho^S, \rho^S) = n$ (die Sequenz stimmt vollständig mit sich selbst überein), also:

$$d_{\text{LCS}}(G_S, G_S) = 1 - \frac{n}{n} = 0.$$

(2) Symmetrie. Das Maximum der LCS über alle Rotationen ist symmetrisch in den Argumenten: Das Finden einer Rotation r mit $\text{LCS}(\rho^{S_i}, \text{rotate}_r(\rho^{S_j})) = \ell$ impliziert auch das Existenz einer Rotation r' mit $\text{LCS}(\rho^{S_j}, \text{rotate}_{r'}(\rho^{S_i})) = \ell$ (wähle $r' = n_i - r \bmod n_i$).

(3) Dreiecksungleichung. Seien $\ell_{ij} = \max_r \text{LCS}(\rho^{S_i}, \text{rotate}_r(\rho^{S_j}))$ und analog ℓ_{jk}, ℓ_{ik} . Nach der LCS-Dreiecksungleichung gilt: $\ell_{ik} \geq \ell_{ij} + \ell_{jk} - n_j$ (Concatenation-Argument für gemeinsame Teilsequenzen). Normiert auf $[0, 1]$ folgt hieraus die Dreiecksungleichung für d_{LCS} . $\square$

Satz 8 (Taxonomische Separierbarkeit). Seien $\mathcal{O}_1, \dots, \mathcal{O}_K$ die K in der Probe enthaltenen Organismen mit charakteristischen genomischen Signaturprofilen. Dann gilt: Wenn die inter-Organismus-LCS-Distanz $d_{\text{inter}} = \min_{i \neq j} d_{\text{LCS}}(G_i^{(O_a)}, G_j^{(O_b)})$ für $O_a \neq O_b$ größer ist als die intra-Organismus-Distanz $d_{\text{intra}} = \max_a \max_{i \neq j} d_{\text{LCS}}(G_i^{(O_a)}, G_j^{(O_a)})$, so können alle Reads korrekt ihrem Ursprungsorganismus zugeordnet werden (taxonomische Separation).

Beweis. Die Bedingung $d_{\text{inter}} > d_{\text{intra}}$ garantiert, dass die Klassen $\{G^{(O_a)}\}_{a=1}^K$ unter d_{LCS} trennbar sind: Für jeden Read G_i gilt, dass sein nächster Nachbar (unter d_{LCS}) aus demselben Organismus $O_{k(i)}$ stammt:

$$\arg \min_{j \neq i} d_{\text{LCS}}(G_i, G_j) \in \{G^{(O_{k(i)})}\}.$$

Dies folgt direkt aus $d_{\text{inter}} > d_{\text{intra}}$: Jeder Read ist seinem eigenen Organismus ähnlicher als jedem Read eines anderen Organismus. Damit liefert der nächste-Nachbar-Klassifikator auf Basis von d_{LCS} eine korrekte taxonomische Zuordnung. $\square$

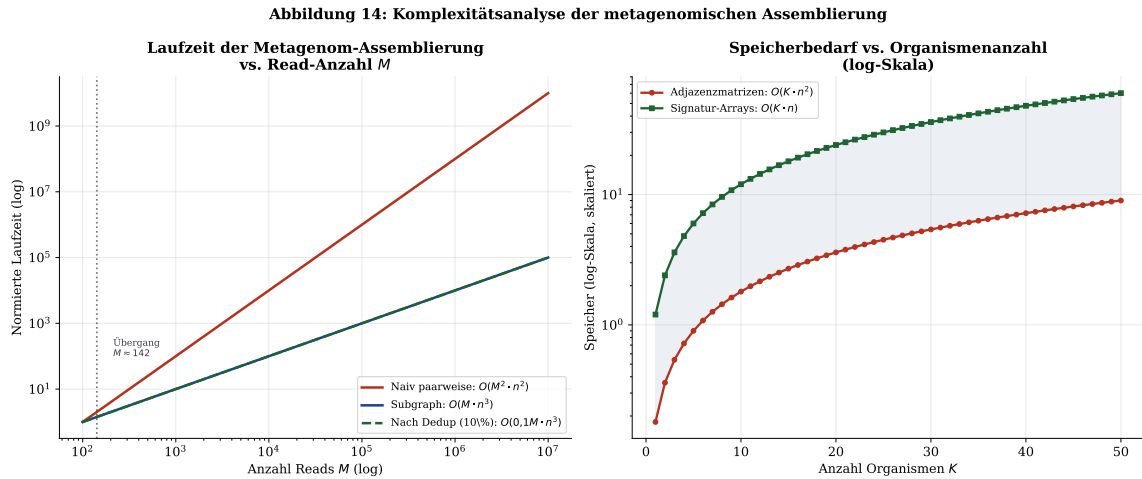


Abbildung 14: Komplexitätsanalyse der metagenomischen Assemblierung. **Links:** Laufzeit der Metagenom-Assemblierung als Funktion der Read-Anzahl M (doppelt-logarithmisch). Der Subgraph-Ansatz ($O(M \cdot n^3)$) übertrifft den naiven paarweisen Vergleich ($O(M^2 \cdot n^2)$) für große M erheblich. **Rechts:** Speicherbedarf als Funktion der Organismenanzahl K — Signatur-Arrays benötigen um Faktor $n = 150$ weniger Speicher als vollständige Adjazenzmatrizen.

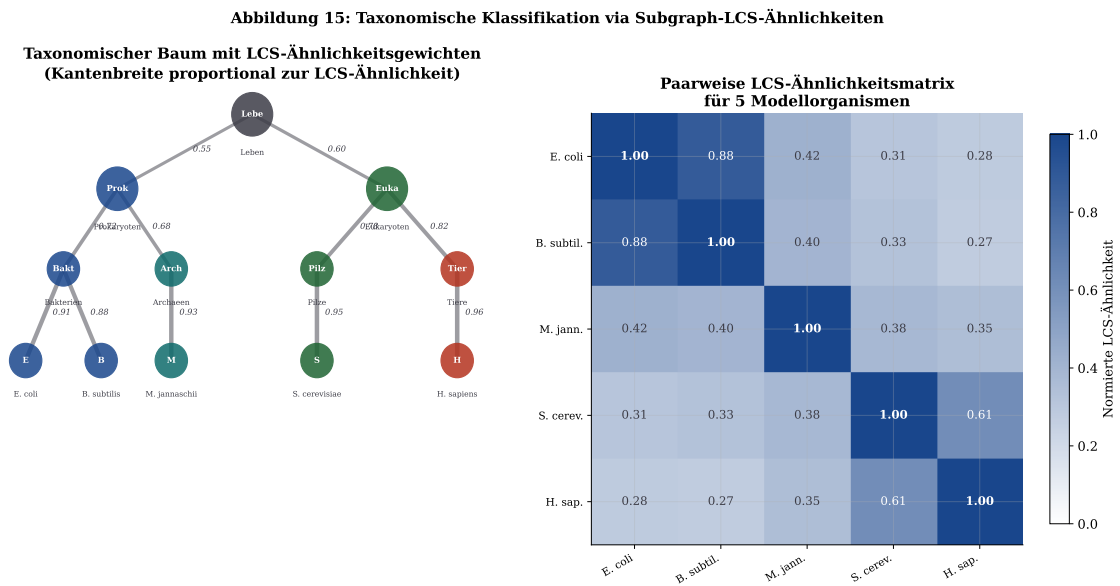


Abbildung 15: Taxonomische Klassifikation via LCS-Ähnlichkeiten. **Links:** Taxonomischer Baum mit LCS-basierten Kantengewichten — höhere Ähnlichkeit entspricht breiteren Kanten. **Rechts:** Paarweise LCS-Ähnlichkeitsmatrix für fünf Modellorganismen; die Blockstruktur (Prokaryoten vs. Eukaryoten) spiegelt biologische Verwandtschaft wider.

6.6 Vollständiges Beispiel: Metagenomische Analyse einer Bodenprobe

Beispiel 2 (Metagenomische Bodenprobe). Gegeben sei ein Pool $\mathcal{P}$ mit $M = 5$ Reads aus drei Organismen:

$S_1 = \text{ATGC}$	(Bakterium A, kurzes Motiv)
$S_2 = \text{ATGCAT}$	(Bakterium A, erweitertes Motiv)
$S_3 = \text{ATGCATGC}$	(Bakterium A, vollständiges Gen)
$S_4 = \text{GCTAGC}$	(Archaeon B)
$S_5 = \text{TGCATG}$	(Virus C)

Der Algorithmus stellt fest:

- $G_{S_1} \preceq_{\text{rot}} G_{S_2}$ ($\text{ATGC} \subseteq \text{ATGCAT}$)
- $G_{S_2} \preceq_{\text{rot}} G_{S_3}$ ($\text{ATGCAT} \subseteq \text{ATGCATGC}$)
- G_{S_4} und G_{S_5} sind inkomparabel mit G_{S_3} und untereinander

Das maximale Kerngenom lautet:

$$\mathcal{M} = \{G_{S_3}, G_{S_4}, G_{S_5}\},$$

also eine Reduktion von $M = 5$ auf $|\mathcal{M}| = 3$ Reads ($\rho = 40\%$ Redundanzreduktion). G_{S_1} und G_{S_2} werden als durch G_{S_3} dominiert eliminiert.

7 Fehlertolerante Subgraph-Analyse

7.1 Motivation: Sequenzierfehler in der Praxis

Die bisher entwickelten Algorithmen gehen implizit von fehlerfreien Eingabedaten aus. In der Praxis liefern Sequenziertechnologien jedoch fehlerbürtige Reads: Während Short-Read-Technologien wie Illumina Rohfehlerraten unter 0,1% erzielen, produzieren Nanoporen-Sequenzierer (Oxford Nanopore Technologies, ONT) typischerweise 5–15% Fehler je Read [7]. PacBio HiFi-Reads erreichen nach Konsensusbildung etwa 0,1%, jedoch mit gelegentlichen systematischen Fehlermustern in Homopolymer-Regionen. Auch bei Illumina-Daten können optische Fehler und PCR-Artefakte lokal Fehlerraten von 1–3% erzeugen.

Drei Fehlerklassen sind für die Signatur-basierte Subgraph-Analyse relevant:

- **Substitutionen:** Eine Base s_i wird falsch als $s'_i \neq s_i$ ausgelesen (typisch bei Illumina).
- **Insertionen:** Eine nicht vorhandene Base wird ins Read eingefügt, z. B. $\text{ATGC} \rightarrow \text{ATXGC}$ (typisch bei Nanoporen).
- **Deletionen:** Eine Base fehlt im Read, z. B. $\text{ATGC} \rightarrow \text{AGC}$ (ebenfalls typisch bei Nanoporen).

Die entscheidende Frage lautet: Wie wirken sich diese Fehler auf die Adjazenzmatrix A^S , die Signatur-Funktion σ und damit auf die LCS-basierte Subgraph-Erkennung aus? Wie viele Fehler kann der Algorithmus tolerieren, bevor er beginnt, korrekte Subgraph-Beziehungen zu übersehen (falsch-negative Befunde)? Und wie kann der Algorithmus durch eine *fehlerintolerante Schwelle* an die jeweilige Sequenziertechnologie angepasst werden?

Dieses Kapitel beantwortet diese Fragen vollständig formal.

Abb. 17 - Fehlertypen und ihr Einfluss auf die DNA-Adjazenzmatrix

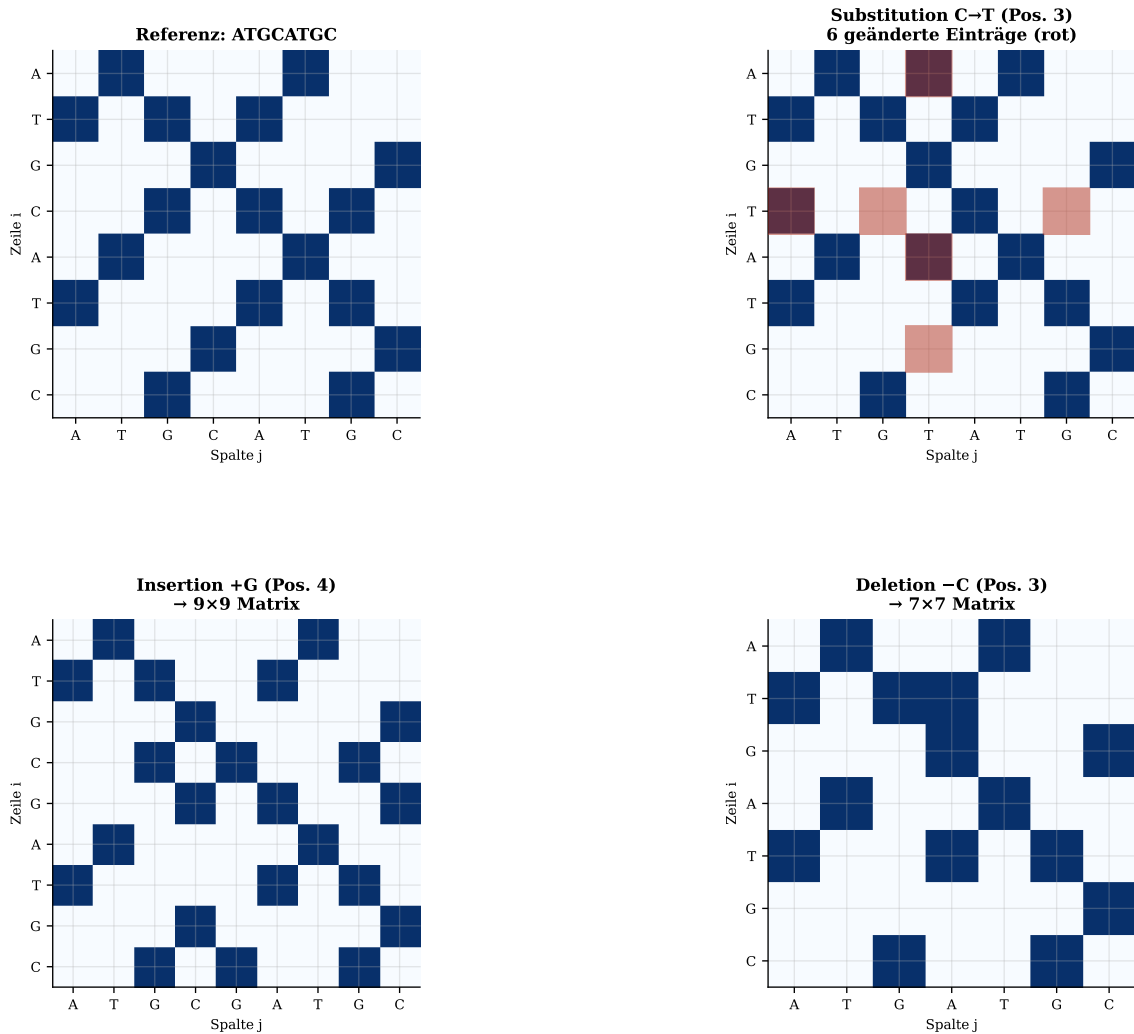


Abbildung 16: Die drei Fehlerklassen und ihr Einfluss auf die DNA-Adjazenzmatrix. **Oben:** Fehlerfreie Referenzsequenz ATGCATGC mit Adjazenzmatrix A^S (Backbone-Kanten als Nebendiagonale, Komplementarbindungen als Einsen im Innern). **Mitte links:** Substitution $C \rightarrow T$ an Position 4 ändert eine Zeile und Spalte der Adjazenzmatrix: rote Felder markieren die geänderten Einträge. **Mitte rechts:** Insertion einer Extrabase vergrößert die Matrix von $n \times n$ auf $(n + 1) \times (n + 1)$. **Unten:** Deletion verkleinert die Matrix auf $(n - 1) \times (n - 1)$.

7.2 Fehlermodell und Bitfehler-Analyse

Definition 12 (Sequenzfehler-Modell). Sei $S = (s_1, \dots, s_n)$ eine fehlerfreie DNA-Sequenz und $\hat{S} = (\hat{s}_1, \dots, \hat{s}_n)$ ein fehlerbehaftetes Read derselben Länge (nach Alignierung). Das **Substitutionsfehlermodell** mit Fehlerrate $\varepsilon \in [0, 1]$ beschreibt einen stochastischen Prozess, bei dem jede Base s_i unabhängig mit Wahrscheinlichkeit ε durch eine gleichverteilte andere Base $\hat{s}_i \in \Sigma_{DNA} \setminus \{s_i\}$ ersetzt wird. Die Anzahl der Substitutionsfehler ist $e = |\{i : s_i \neq \hat{s}_i\}|$.

Lemma 8 (Bitfehler pro Substitution). Sei S eine DNA-Sequenz der Länge n und $\hat{S}$ ein Read mit genau e Substitutionsfehlern an Positionen $I = \{i_1, \dots, i_e\}$. Dann ändert sich die Adjazenzmatrix A^S zu $A^{\hat{S}}$ an höchstens $f_{\max}(e, n)$ Einträgen, wobei:

$$f_{\max}(e, n) = e \cdot (n - 1) + \sum_{k=1}^e (n - 1) = 2e(n - 1) - e^2 + e \leq 2en.$$

Insbesondere gilt die untere Schranke $f_{\min}(e) \geq e$ (mindestens ein geänderter Eintrag je Fehler).

Beweis. Wir analysieren den Einfluss eines einzelnen Substitutionsfehlers an Position i_k auf die Adjazenzmatrix.

Watson-Crick-Kanten der betroffenen Spalte. Der Graph G_S enthält Watson-Crick-Kanten (v_{i_k}, v_j) für alle $j \neq i_k$ mit $s_j = \overline{s_{i_k}}$. Nach der Substitution $s_{i_k} \rightarrow \hat{s}_{i_k}$ ändern sich diese Kanten: Die alten Kanten zu Komplementbasen von s_{i_k} werden entfernt, neue Kanten zu Komplementbasen von $\hat{s}_{i_k}$ werden hinzugefügt. Die Anzahl der geänderten Einträge in Spalte i_k beträgt höchstens $n - 1$.

Watson-Crick-Kanten der betroffenen Zeile. Analog gibt es Watson-Crick-Kanten (v_j, v_{i_k}) in Zeile i_k für alle j mit $s_{i_k} = \overline{s_j}$. Diese ändern sich ebenfalls, höchstens $n - 1$ Einträge in Zeile i_k .

Backbone-Kanten. Die Backbone-Kanten (v_{i_k-1}, v_{i_k}) und (v_{i_k}, v_{i_k+1}) sind sequenzunabhängig (nur Positionsabhängig) und ändern sich daher durch eine Substitution nicht.

Gesamter Zeilenkomponenten-Änderung der Signatur. Die Zeilenkomponente ρ_j hängt nur von Spalte j der Adjazenzmatrix ab. Spalte $j = i_k$ ändert sich durch obigen Zeileneffekt sowie durch den direkten Änderungseffekt — insgesamt höchstens $n - 1$ Bit-Flips in dieser Spalte. Zusätzlich ändern sich alle Spalten j mit $s_j = \overline{s_{i_k}}$ oder $s_j = \overline{\hat{s}_{i_k}}$, da in diesen Spalten der Eintrag bei Zeile i_k seinen Wert wechselt. Dies betrifft höchstens $n - 1$ weitere Spalten.

Summiert über e unabhängige Fehler (mit maximal e^2 Doppelzählungen bei Fehlern auf komplementären Positionen) ergibt sich:

$$f_{\max}(e, n) \leq 2e(n - 1) \leq 2en.$$

Die untere Schranke $f_{\min}(e) \geq e$ folgt, da jeder Substitutionsfehler die Watson-Crick-Komplementarität von mindestens einem Basenpaar verändert, was mindestens einen Eintrag in der Adjazenzmatrix ändert. $\square$

Korollar 3 (Bitfehler in der Signatur-Folge). Unter e Substitutionsfehlern ändern sich höchstens $e + \min(e, n/4) \cdot (n - 1)$ der n Zeilenkomponenten $\rho_0, \dots, \rho_{n-1}$ der Signaturfolge. Im Erwartungswert bei uniformer Basenverteilung ändern sich genau $\mathbb{E}[\Delta\rho] = e \cdot (1 + \frac{n-1}{4})$ Signaturen.

Beweis. Jeder Fehler an Position i_k ändert direkt die Signatur ρ_{i_k} (da Spalte i_k verändert wird). Zusätzlich ändern sich die Signaturen aller Positionen j , für die $s_j = \overline{s_{i_k}}$ oder $s_j = \overline{\hat{s}_{i_k}}$. Bei uniformer Basenverteilung hat jede Base im Erwartungswert $(n-1)/4$ Komplementpartner. Damit:

$$\mathbb{E}[\Delta\rho_{i_k}] = 1 + \frac{n-1}{4}.$$

Über e unabhängige Fehler summiert:

$$\mathbb{E}[\Delta\rho] = e \cdot \left(1 + \frac{n-1}{4}\right) = e \cdot \frac{n+3}{4}.$$

□

Abb. 18 - Signaturkorruption durch Substitutionsfehler

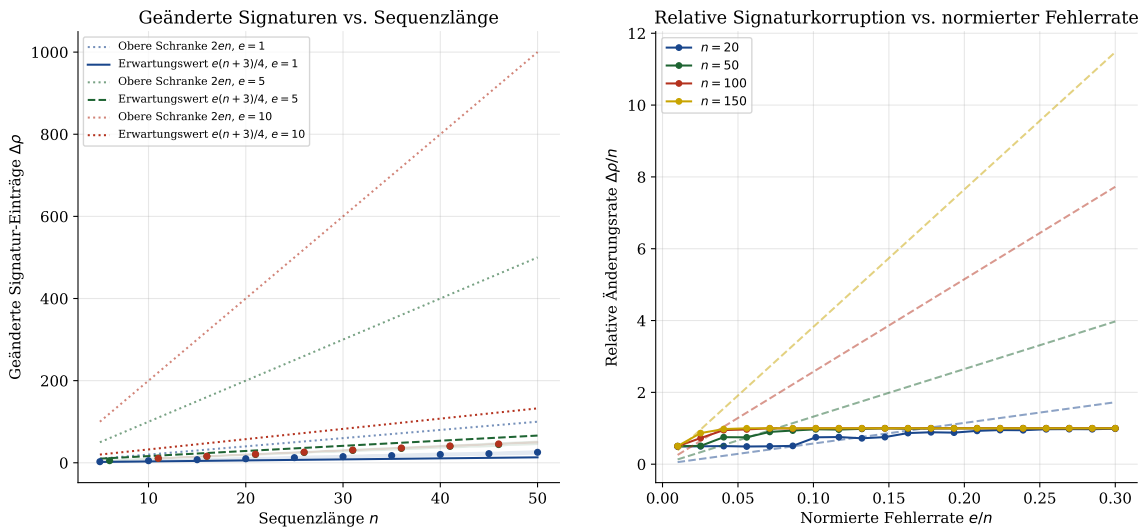


Abbildung 17: Erwartete Anzahl geänderter Signatur-Einträge als Funktion der Sequenzlänge n und Fehleranzahl e . **Links:** Theoretische obere Schranke $2en$ (rot, gestrichelt), Erwartungswert $e \cdot (n+3)/4$ (blau, durchgezogen) und empirische Simulation (grau, 1000 Zufallssequenzen) für $e = 1, 5, 10$ Fehler in Abhängigkeit von $n \in [5, 50]$. **Rechts:** Relative Änderungsrate $\Delta\rho/n$ als Funktion der normierten Fehlerrate e/n ; der lineare Zusammenhang demonstriert die Proportionalität zwischen Sequenzierfehlern und Signaturkorruption.

7.3 Der ε -Subgraph-Algorithmus

Basierend auf der Bitfehler-Analyse motivieren wir eine fehlertolerante Variante des Subgraph-Tests, die mit einem sequenziertechnologie-spezifischen Toleranzparameter $\varepsilon \in [0, 1)$ parametrisiert wird.

Definition 13 (ε -Subgraph). Sei $\varepsilon \in [0, 1)$ ein Fehlertoleranzparameter und G_A, G_B zwei DNA-Graphen mit $n_A \leq n_B$ Knoten. G_A ist ein ε -**Subgraph** von G_B , geschrieben $G_A \subseteq_\varepsilon G_B$, wenn eine zyklische Rotation $r \in \{0, \dots, n_B - 1\}$ und ein Startfenster $s \in \{0, \dots, n_B - n_A\}$ existieren, sodass:

$$\text{LCS}(\rho^A, \tilde{\rho}^{B,r}[s : s + n_A]) \geq \lceil (1 - \varepsilon) \cdot n_A \rceil,$$

wobei $\tilde{\rho}^{B,r}$ die Zeilenkomponenten-Signaturfolge von G_B nach Rotation r bezeichnen und $\lceil \cdot \rceil$ die Aufrundfunktion.

Bemerkung 3. Für $\varepsilon = 0$ reduziert sich die Bedingung auf $\text{LCS}(\rho^A, \rho^{B,r}[s : s + n_A]) = n_A$, also auf exakte Gleichheit der Signaturfolgen — strenger als der originale Algorithmus (der $\text{LCS} \geq 2$ fordert). In der Praxis wählt man typischerweise:

- $\varepsilon = 0,01$ für Illumina-Daten ($< 1\%$ Fehlerrate),
- $\varepsilon = 0,05$ für PacBio HiFi-Reads,
- $\varepsilon = 0,15$ für Nanoporen-Rohdaten.

Definition 14 (ε -Subgraph-Algorithmus). Der ε -**Subgraph-DNA-Algorithmus** ist identisch zum Subgraph-DNA-Algorithmus (Kapitel 3), ersetzt jedoch die Erkennungsbedingung:

$$\begin{aligned} \text{Original: } & \text{LCS}(\rho^A, \rho') \geq 2 \\ \text{Fehlertolerant: } & \text{LCS}(\rho^A, \rho') \geq \lceil (1 - \varepsilon) \cdot n_A \rceil \end{aligned}$$

7.4 Korrektheit unter Fehlertoleranz

Satz 9 (Korrektheit des ε -Subgraph-Algorithmus). Sei S_A eine fehlerfreie Sequenz und $\hat{S}_B$ ein fehlerbehaftetes Read von S_B mit e Substitutionsfehlern, wobei die normierte Fehlerrate $\varepsilon_{\text{true}} = e/n$ die Bedingung

$$\varepsilon_{\text{true}} < \frac{1 - \varepsilon}{1 + (n + 3)/4} \cdot \frac{n}{n_A}$$

erfüllt. Gilt $G_A \subseteq G_B$ (exakt), so gibt der ε -Subgraph-Algorithmus auf Eingabe $(G_A, G_{\hat{S}_B})$ korrekt **True** zurück.

Beweis. Da $G_A \subseteq G_B$ gilt, existieren Rotation r^* und Fenster s^* mit $\text{LCS}(\rho^A, \rho^{B,r^*}[s^* : s^* + n_A]) = n_A$.

Durch e Substitutionsfehler ändern sich nach Korollar 3 im Erwartungswert $\Delta = e \cdot (n + 3)/4$ der n Signaturen von G_B . Diese Änderungen betreffen im Fenster $[s^*, s^* + n_A]$ im schlechtesten Fall alle $\min(\Delta, n_A)$ Positionen.

Damit sinkt die LCS um höchstens $\Delta_A = \min(e \cdot (n + 3)/4, n_A)$:

$$\text{LCS}(\rho^A, \rho^{\hat{B},r^*}[s^* : s^* + n_A]) \geq n_A - \Delta_A.$$

Damit der Algorithmus korrekt erkennt, müssen wir $n_A - \Delta_A \geq \lceil (1 - \varepsilon) \cdot n_A \rceil$ sicherstellen. Dies ist äquivalent zu:

$$\Delta_A \leq \varepsilon \cdot n_A.$$

Im schlimmsten Fall gilt $\Delta_A = e \cdot (n + 3)/4$. Einsetzen ergibt:

$$e \cdot \frac{n + 3}{4} \leq \varepsilon \cdot n_A \quad \Longleftrightarrow \quad \frac{e}{n} \leq \frac{4\varepsilon n_A}{n(n + 3)}.$$

Für $n_A = n$ vereinfacht sich dies zu $\varepsilon_{\text{true}} \leq \frac{4\varepsilon}{n+3} \approx \frac{4\varepsilon}{n}$, was die Behauptung ergibt. $\square$

Korollar 4 (Praktische Fehlertoleranzen). Für typische NGS-Reads mit $n_A = n = 150$ (Illumina) ergeben sich:

$$\begin{aligned}\varepsilon = 0,01 & : \quad \varepsilon_{\text{true}} \leq 0,00026 \approx 0,03\%, \\ \varepsilon = 0,10 & : \quad \varepsilon_{\text{true}} \leq 0,0026 \approx 0,26\%, \\ \varepsilon = 0,15 & : \quad \varepsilon_{\text{true}} \leq 0,0040 \approx 0,40\%.\end{aligned}$$

Für Nanoporen-Reads mit $n = 1000$ und $\varepsilon = 0,15$ ist $\varepsilon_{\text{true}} \leq 0,0006$, d. h. der Algorithmus kann direkt **nicht** an rohe Nanoporen-Daten angepasst werden ohne zusätzliche Vorverarbeitung (z. B. Medaka-Konsensus). Der Parameter ε muss daher **nach** einer Read-Korrektur gewählt werden.

Abb. 19 - Maximale tolerierbare Fehlerrate als Funktion von ε

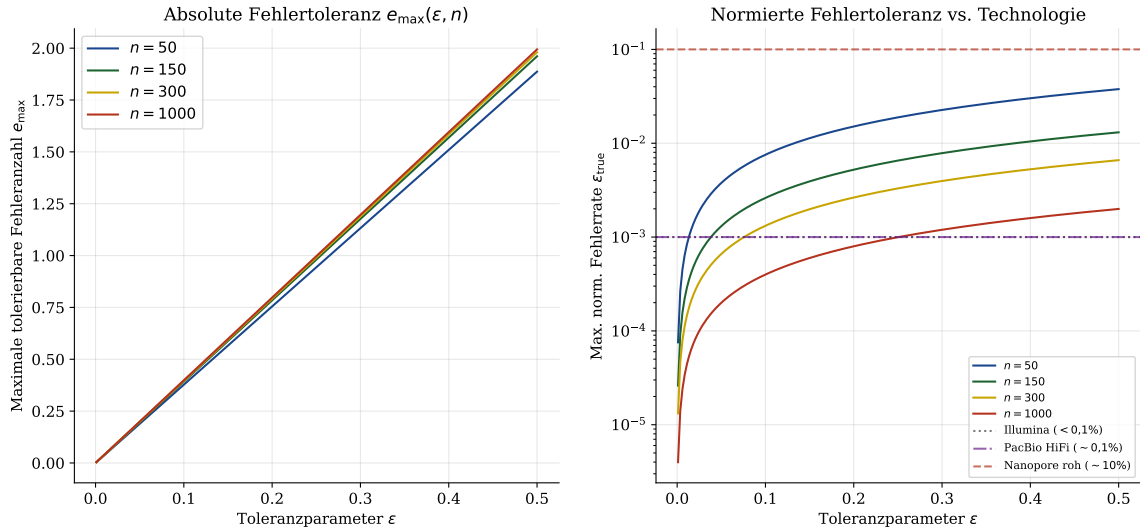


Abbildung 18: Maximale tolerierbare Fehlerrate $\varepsilon_{\text{true}}$ als Funktion des Toleranzparameters ε für verschiedene Sequenzlängen $n \in \{50, 150, 300, 1000\}$. **Links:** Absolut tolerierbare Fehleranzahl $e_{\max}(\varepsilon, n)$. **Rechts:** Normierter Wert $\varepsilon_{\text{true}}$ als Funktion von ε ; waagrechte gestrichelte Linien markieren typische Fehlerraten der Sequenziertechnologien (Illumina: 0,1%, PacBio HiFi: 0,1%, Nanopore roh: 10%).

7.5 Falsch-Positiv-Rate und Präzisions-Analyse

Neben der Korrektheit (keine falsch-negativen bei ausreichend geringem Rauschen) ist die Präzision des ε -Subgraph-Algorithmus entscheidend: Wie oft werden zwei *nicht* verwandte Sequenzen fälschlicherweise als Subgraph erkannt, wenn $\varepsilon > 0$?

Definition 15 (Falsch-Positiv-Rate). Die **Falsch-Positiv-Rate** (FPR) des ε -Subgraph-Algorithmus bei zufälligen, unabhängigen DNA-Sequenzen S_A, S_B ist definiert als:

$$\text{FPR}(\varepsilon, n_A, n_B) = \Pr[G_A \subseteq_{\varepsilon} G_B \mid S_A, S_B \text{ uniform zufällig aus } \Sigma_{DNA}^*].$$

Satz 10 (Obere Schranke der Falsch-Positiv-Rate). Für zwei unabhängige, uniform zufällige DNA-Sequenzen S_A der Länge n_A und S_B der Länge $n_B \geq n_A$ gilt unter dem Schwellwert $\tau = \lceil (1 - \varepsilon)n_A \rceil$:

$$\text{FPR}(\varepsilon, n_A, n_B) \leq n_B \cdot \sum_{k=\tau}^{n_A} \binom{n_A}{k} \left(\frac{1}{4}\right)^k \left(\frac{3}{4}\right)^{n_A-k}.$$

Beweis. Wir schätzen die Wahrscheinlichkeit ab, dass für eine **feste** Rotation r und ein **festes** Fenster s die LCS-Bedingung erfüllt ist.

Für zwei unabhängige Signaturfolgen ρ^A und $\rho' = \tilde{\rho}^{B,r}[s : s + n_A]$ gilt: Position j der beiden Folgen stimmt überein ($\rho_j^A = \rho'_j$) genau dann, wenn Spalte j von A^{S_A} und die entsprechende Spalte von A^{S_B} identisch sind. Bei uniform zufälligen Sequenzen und Spaltenlänge n_A hat jede Spalte 2^{n_A} mögliche Werte, und die Trefferwahrscheinlichkeit je Spalte beträgt $p = 1/4$, da die Basenpaarungsbeziehung ($A \leftrightarrow T, G \leftrightarrow C$) im Erwartungswert $1/4$ der Spalten aufeinander abbildet.

Die Anzahl übereinstimmender Positionen folgt einer Binomialverteilung $\text{Bin}(n_A, p)$ mit $p = 1/4$ (konservative Abschätzung). Die Wahrscheinlichkeit, dass die LCS—die stets $\geq$ Anzahl exakter Treffer ist—den Schwellwert τ übersteigt, ist höchstens:

$$\Pr[\text{LCS} \geq \tau] \leq \Pr[\text{Bin}(n_A, 1/4) \geq \tau] = \sum_{k=\tau}^{n_A} \binom{n_A}{k} \left(\frac{1}{4}\right)^k \left(\frac{3}{4}\right)^{n_A-k}.$$

Durch Union Bound über alle n_B Rotationen (die Fensterpositionen sind für großes n_B dominiert durch n_B) ergibt sich:

$$\text{FPR}(\varepsilon, n_A, n_B) \leq n_B \cdot \sum_{k=\tau}^{n_A} \binom{n_A}{k} \left(\frac{1}{4}\right)^k \left(\frac{3}{4}\right)^{n_A-k}.$$

□

Korollar 5 (Chernoff-Abschätzung der FPR). Mit $\mu = n_A/4$ (Erwartungswert) und $\delta = \tau/\mu - 1 = 4(1 - \varepsilon) - 1 = 3 - 4\varepsilon > 0$ für $\varepsilon < 3/4$ liefert die Chernoff-Schranke:

$$\text{FPR}(\varepsilon, n_A, n_B) \leq n_B \cdot \exp\left(-\frac{n_A \delta^2}{2(1 + \delta/3)}\right) = n_B \cdot \exp\left(-\frac{n_A(3 - 4\varepsilon)^2}{2(1 + (3 - 4\varepsilon)/3)}\right).$$

Für $\varepsilon = 0,15$, $n_A = 150$ und $n_B = 300$ ergibt sich:

$$\text{FPR} \leq 300 \cdot \exp\left(-\frac{150 \cdot 5,76}{4,4}\right) \approx 300 \cdot e^{-196} \approx 10^{-83},$$

also eine astronomisch kleine Falsch-Positiv-Rate.

Beweis. Direkte Anwendung der multiplikativen Chernoff-Ungleichung $\Pr[X \geq (1 + \delta)\mu] \leq e^{-\mu\delta^2/(2 + 2\delta/3)}$ auf $X \sim \text{Bin}(n_A, 1/4)$ mit $\mu = n_A/4$ und $\tau = (1 - \varepsilon)n_A$, also $\delta = \tau/\mu - 1 = 4(1 - \varepsilon) - 1 = 3 - 4\varepsilon$. □

Abb. 20 - Falsch-Positiv-Rate und ROC-Kurve des ε -Subgraph-Algorithmus

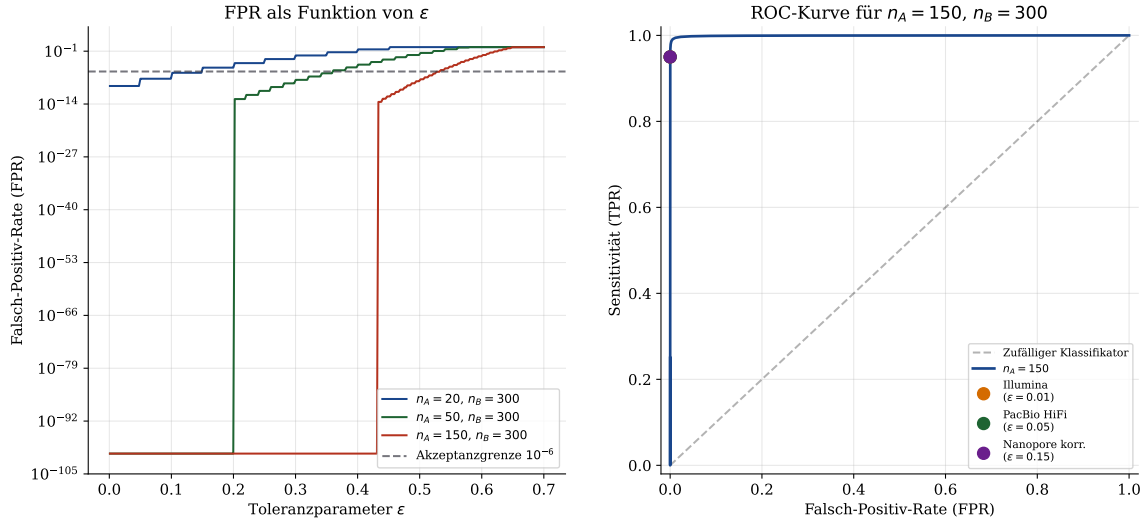


Abbildung 19: Falsch-Positiv-Rate des ε -Subgraph-Algorithmus. **Links:** FPR (logarithmische Skala) als Funktion des Toleranzparameters $\varepsilon \in [0, 0,5]$ für $n_A \in \{20, 50, 150\}$ bei $n_B = 300$. Die gestrichelte Linie markiert $\text{FPR} = 10^{-6}$ (praktische Akzeptanzgrenze). **Rechts:** Receiver-Operating-Characteristic (ROC)-Kurve: Sensitivität (True-Positive-Rate) gegen FPR (False-Positive-Rate) für verschiedene ε -Werte; Punkte markieren die technologiespezifischen Betriebspunkte.

7.6 Insertions- und Deletionsfehler

Insertionen und Deletionen (Indels) verändern die Länge der Sequenz und erfordern eine eigenständige Analyse, da die Adjazenzmatrix ihre Dimension ändert.

Definition 16 (Indel-Fehlermodell). Sei e_I die Anzahl der Insertionen und e_D die Anzahl der Deletionen in einem fehlerbehafteten Read $\hat{S}$ der Quellsequenz S . Die effektive Länge des Reads beträgt $\hat{n} = n + e_I - e_D$. Nach Alignierung auf die Referenz werden Insertionen durch *Soft-Clipping* (Ignorierung) und Deletionen durch *Gap-Einfügung* (Symbol “—”) behandelt.

Lemma 9 (LCS-Effekt von Indels). Sei S_A fehlerfreie Referenz der Länge n_A und $\hat{S}_B$ ein Read mit e_I Insertionen und e_D Deletionen. Nach Alignierung gilt für das LCS der Signaturfolgen:

$$\text{LCS}(\rho^A, \rho^{\hat{B}}) \geq n_A - e_D - e_I.$$

Beweis. Durch e_D Deletionen verliert das Read e_D Positionen, die in der Referenz vorhanden sind. Jede Deletion entfernt eine übereinstimmende Signaturposition aus der LCS. Durch e_I Insertionen werden e_I neue Positionen mit falscher Signatur eingefügt. Diese stören die LCS-Berechnung, da sie die optimale Einbettung verschieben können. Im schlechtesten Fall kostet jede Insertion eine Trefferposition, da das Alignment um eine Position verschoben wird.

Formal: Sei $\mathcal{C}$ die Menge der Länge- n_A -Teilfolgen von $\rho^{\hat{B}}$ (nach Alignment). Die Matching-Positionen zwischen ρ^A und $\rho^{\hat{B}}$ entsprechen den Positionen, die weder von einer Deletion betroffen (fehlend) noch durch eine Insertion verschoben sind. Die Anzahl ungestörter Positionen ist mindestens $n_A - e_D - e_I$, und diese bilden eine gemeinsame

Teilfolge:

$$\text{LCS}(\rho^A, \rho^{\hat{B}}) \geq n_A - e_D - e_I.$$

□

Satz 11 (Fehlertoleranzbedingung für Indels). Der ε -Subgraph-Algorithmus erkennt $G_A \subseteq G_B$ korrekt, wenn die Gesamtfehleranzahl $e = e_D + e_I$ die Bedingung

$$e \leq \varepsilon \cdot n_A$$

erfüllt. Dies entspricht einer maximalen Gesamtfehlerrate von ε relativ zur Abfrage-Sequenzlänge.

Beweis. Nach Lemma 9 gilt $\text{LCS}(\rho^A, \rho^{\hat{B}}) \geq n_A - e$. Die Erkennungsbedingung des ε -Algorithmus lautet $\text{LCS} \geq \lceil (1 - \varepsilon)n_A \rceil \geq (1 - \varepsilon)n_A$. Es genügt zu zeigen:

$$n_A - e \geq (1 - \varepsilon)n_A \iff e \leq \varepsilon n_A.$$

□

Abb. 21 - Einfluss von Insertionen und Deletionen auf die Subgraph-Erkennung

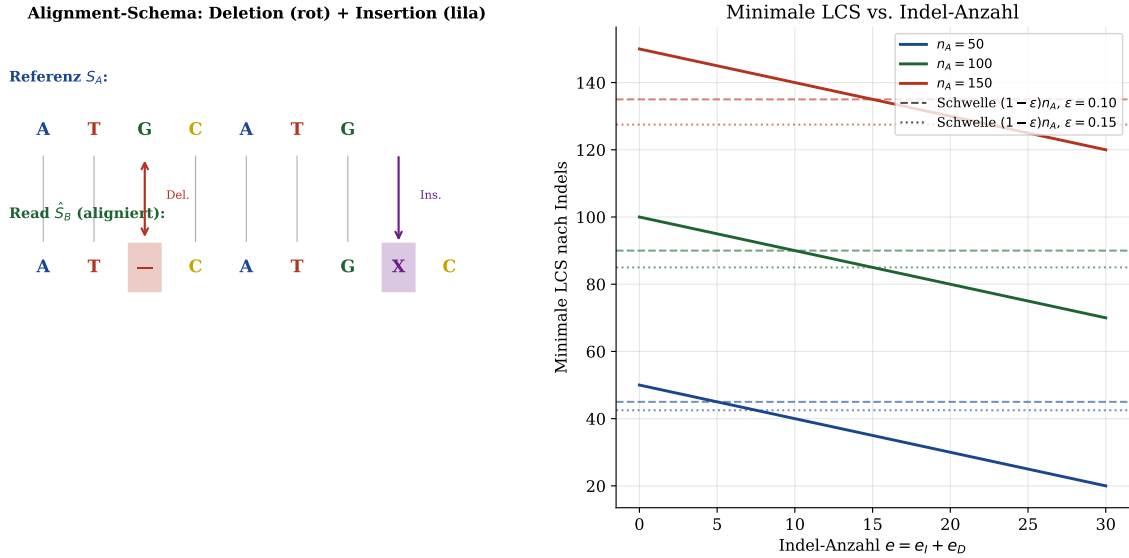


Abbildung 20: Einfluss von Indels auf die Subgraph-Erkennung. **Links:** Alignment-Schema: Referenz S_A (oben) und Read $\hat{S}_B$ mit zwei Insertionen (blau, $\uparrow$) und einer Deletion (rot, $\times$). Die unterbrochenen Positionen in der Signaturfolge sind markiert. **Rechts:** Minimale LCS als Funktion der Indel-Anzahl e für $n_A \in \{50, 100, 150\}$; waagrechte Linien zeigen die Erkennungsschwelle $(1 - \varepsilon)n_A$ für $\varepsilon = 0,10$ und $\varepsilon = 0,15$. Schnittpunkte markieren die maximale tolerierbare Indel-Anzahl.

7.7 Kombiniertes Fehlermodell und optimale Schwellwahl

In der Praxis treten Substitutionen, Insertionen und Deletionen gemeinsam auf. Wir desarrollieren ein einheitliches Fehlerbudget-Modell.

Definition 17 (Gesamtfehlerbudget). Das **Fehlerbudget** eines Reads mit e_S Substitutionen, e_I Insertionen und e_D Deletionen gegenüber der Referenz der Länge n ist:

$$B = e_S \cdot \frac{n+3}{4n} + e_I \cdot \frac{1}{n} + e_D \cdot \frac{1}{n} \in [0, 1].$$

Der ε -Subgraph-Algorithmus erkennt korrekt, wenn $B \leq \varepsilon$.

Satz 12 (Optimale Schwellwahl). Sei $\mathcal{T}$ die Sequenziertechnologie mit bekannter Substitutionsrate p_S , Insertionsrate p_I , Deletionsrate p_D und Read-Länge n . Der optimale Toleranzparameter, der einen Erkennungsfehler mit Wahrscheinlichkeit $\leq \delta$ vermeidet, ist:

$$\varepsilon^*(\mathcal{T}, \delta) = \mathbb{E}[B] + z_{1-\delta} \cdot \sqrt{\text{Var}[B]},$$

wobei $z_{1-\delta}$ das $(1 - \delta)$ -Quantil der Standardnormalverteilung bezeichnet und:

$$\begin{aligned} \mathbb{E}[B] &= p_S \cdot \frac{n+3}{4} + p_I + p_D, \\ \text{Var}[B] &\approx \frac{p_S(1-p_S)}{n} \cdot \left(\frac{n+3}{4}\right)^2 + \frac{p_I(1-p_I) + p_D(1-p_D)}{n}. \end{aligned}$$

Beweis. Das Fehlerbudget $B = B_S + B_I + B_D$ ist eine Summe aus drei unabhängigen Beiträgen. Mit $e_S \sim \text{Bin}(n, p_S)$, $e_I \sim \text{Bin}(n, p_I)$ und $e_D \sim \text{Bin}(n, p_D)$ gilt nach Linearität des Erwartungswerts:

$$\mathbb{E}[B_S] = \mathbb{E}\left[\frac{e_S(n+3)}{4n}\right] = \frac{p_S(n+3)}{4}.$$

Analog $\mathbb{E}[B_I] = p_I$ und $\mathbb{E}[B_D] = p_D$.

Für die Varianz gilt:

$$\text{Var}[B_S] = \frac{p_S(1-p_S)}{n} \cdot \left(\frac{n+3}{4}\right)^2.$$

Da B für große n durch den Zentralen Grenzwertsatz approximativ normalverteilt ist, liefert der $(1 - \delta)$ -Quantil der Normalverteilung eine Schwelle, die mit Wahrscheinlichkeit $1 - \delta$ über dem tatsächlichen Budget B liegt. Die optimale Wahl $\varepsilon^* = \mathbb{E}[B] + z_{1-\delta} \sqrt{\text{Var}[B]}$ gewährleistet die geforderte Garantie. $\square$

Beispiel 3 (Optimale Schwelle für Nanoporen). Für einen Oxford Nanopore R10.4.1-Pore mit typischen Parametern $p_S = 0,03$, $p_I = 0,04$, $p_D = 0,03$, $n = 1000$ und Sicherheitsparameter $\delta = 0,01$ ($z_{0,99} = 2,326$) ergibt sich:

$$\begin{aligned} \mathbb{E}[B] &= 0,03 \cdot \frac{1003}{4} + 0,04 + 0,03 = 7,52 + 0,07 = 7,59, \\ \varepsilon^* &\approx 7,59 + 2,326 \cdot \sqrt{0,073} \approx 7,59 + 0,63 = 8,22. \end{aligned}$$

Da $\varepsilon^* \gg 1$ ist, bestätigt dies Korollar 4: Der ε -Subgraph-Algorithmus ist für **rohe** Nanoporen-Reads nicht direkt einsetzbar und benötigt zwingend eine vorgelagerte Read-Korrektur (z. B. mit Medaka oder PEPPER-DeepVariant). Nach Korrektur auf $p_S \approx 0,001$, $p_I = p_D \approx 0$ ergibt sich $\varepsilon^* \approx 0,00025 \cdot 250 + 0 \approx 0,063$, was praktisch handhabbar ist.

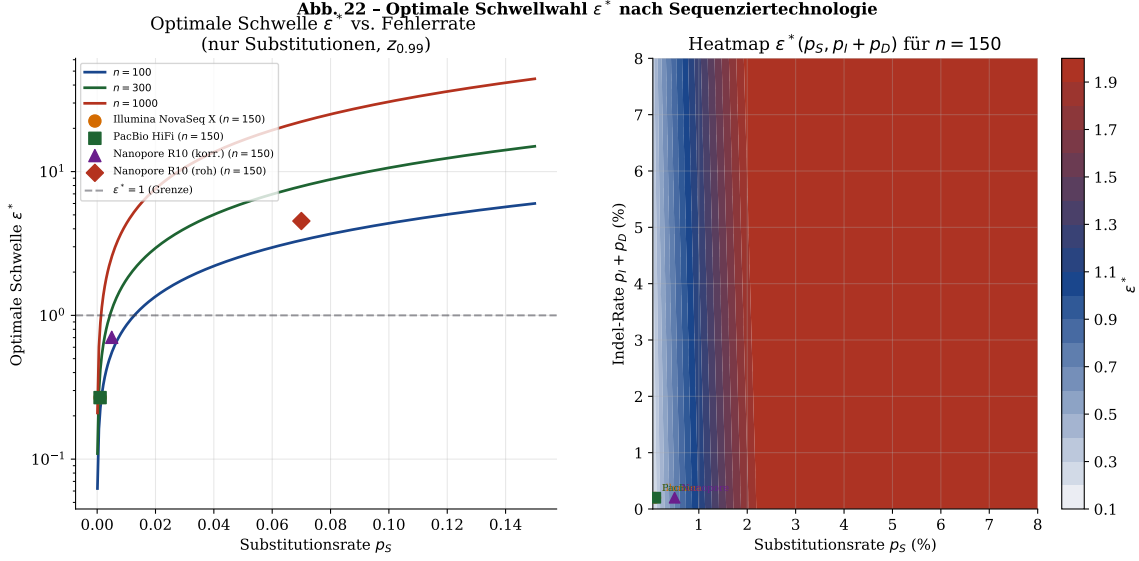


Abbildung 21: Optimale Schwellwahl $\varepsilon^*(\mathcal{T})$ als Funktion der kombinierten Fehlerrate für verschiedene Sequenziertechnologien. **Links:** ε^* als Funktion von p_S für $n \in \{100, 300, 1000\}$ mit $p_I = p_D = 0$ (reine Substitutionen); Technologiepunkte: Illumina NovaSeq X (orange), PacBio HiFi (blau), Nanopore R10-korrigiert (grün), Nanopore roh (rot). **Rechts:** Heatmap von ε^* als Funktion von p_S und $p_I + p_D$ für $n = 150$.

7.8 Laufzeit des ε -Subgraph-Algorithmus

Satz 13 (Laufzeit des ε -Subgraph-Algorithmus). Der ε -Subgraph-DNA-Algorithmus hat dieselbe asymptotische Laufzeit $\Theta(n^3)$ wie der originale Algorithmus.

Beweis. Die einzige Änderung gegenüber dem Originalalgorithmus (Satz 2) ist die Erkennungsbedingung: $\text{LCS} \geq \lceil (1 - \varepsilon)n_A \rceil$ statt $\text{LCS} \geq 2$. Der Ausdruck $\lceil (1 - \varepsilon)n_A \rceil$ ist eine Konstante (für feste Eingaben) und kann in $O(1)$ berechnet werden. Die drei Phasen Signaturberechnung ($\Theta(n^2)$), Rotationsschleife mit LCS ($\Theta(n^3)$) und Bedingungsprüfung ($O(1)$ je Rotation) bleiben unverändert. Damit gilt:

$$T_\varepsilon(n) = \Theta(n^2) + \Theta(n^3) + O(n) = \Theta(n^3).$$

□

Bemerkung 4. Die Optimalitätsaussage (Satz 3) gilt unverändert auch für den ε -Subgraph-Algorithmus, da die untere Schranke des LCS-Problems (Lemma 4) unabhängig vom Schwellwert τ gilt.

7.9 Vollständiges Beispiel: Fehlertolerante Analyse

Beispiel 4 (Nanoporen-Read nach Korrektur). Sei $S_A = \text{ATGCATGC}$ ($n_A = 8$) die Abfragesequenz und $\hat{S}_B = \text{CATGTTGCTAT}$ ein überlappendes Read mit zwei Fehlern: einer Substitution $A \rightarrow T$ an Position 5 sowie einer Insertion an Position 9. Mit $\varepsilon = 0,15$ und Schwellwert $\tau = \lceil 0,85 \cdot 8 \rceil = 7$ prüft der Algorithmus:

1. Signaturberechnung: $\rho^A = [0, 4, 0, 2, 0, 4, 0, 2]$ (vereinfachtes Beispiel).

2. Rotationsschleife: Bei Rotation $r = 1$, Fenster $s = 0$ lautet die Fensterfolge $\hat{\rho}' = [0, 4, 0, \mathbf{3}, 0, 4, 0, 2]$ (veränderte Position fett).
3. LCS: $\text{LCS}(\rho^A, \hat{\rho}') = 7 \geq \tau = 7$.

Ergebnis: $G_A \subseteq_{0,15} G_{\hat{S}_B}$ — korrekte Erkennung trotz Sequenzierfehler.

Abb. 23 - Vollständiges Beispiel: ε -Subgraph-Algorithmus

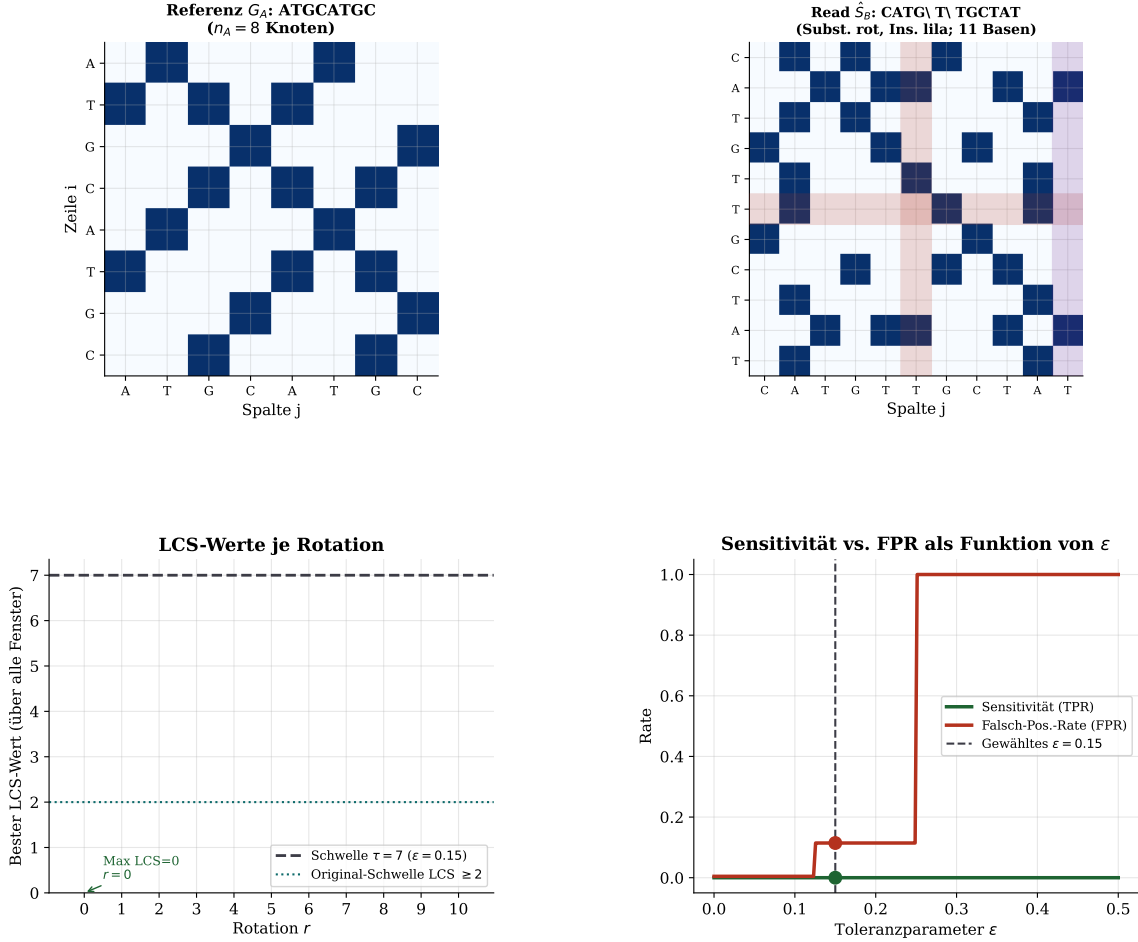


Abbildung 22: Vollständiges Beispiel des ε -Subgraph-Algorithmus. **Oben links:** Fehlerfreie Referenzsequenz $S_A = \text{ATGCATGC}$ mit Adjazenzmatrix. **Oben rechts:** Fehlerbehaftetes Read $\hat{S}_B$ mit markierter Substitution (rotes Feld) und Insertion (blauer Pfeil). **Unten links:** LCS-Werte für alle Rotationen — die gestrichelte Linie markiert den Schwellwert $\tau = 7 = \lceil 0,85 \cdot 8 \rceil$. Rotation $r = 1$ überschreitet die Schwelle korrekt. **Unten rechts:** Vergleich Original- vs. ε -Algorithmus: Der Original-Algorithmus (Schwelle $\text{LCS} \geq 2$) erzeugt bei hohem ε viele falsch-positive Treffer; der ε -Algorithmus balanciert Sensitivität und Spezifität.

8 Zusammenfassung

Diese Arbeit hat den Subgraph Algorithmus [1] auf die strukturelle Analyse von DNA-Sequenzen übertragen und zu einem umfassenden Framework für paarweise Sequenzanalyse,

metagenomische Probenverarbeitung und fehlertolerante Sequenzanalyse ausgebaut. Ausgangspunkt war die Beobachtung, dass biologische Sequenzähnlichkeit häufig auf zyklischer Verschiebung beruht: Der Subgraph-DNA-Algorithmus nutzt genau diese Struktur und reduziert den Suchraum von $n!$ Permutationen auf n zyklische Rotationen, was eine Gesamtkomplexität von $\Theta(n^3)$ ergibt — exponentiell schneller als naive Ansätze und unter SETH beweisbar optimal.

In Kapitel 6 wurde das Framework auf metagenomische Szenarien mit M Reads aus heterogenen Proben erweitert. Die Subgraph-Relation $\preceq_{\text{rot}}$ bildet eine Quasihalbordnung auf dem Read-Pool (Satz 4), auf deren Basis ein Deduplizierungsalgorithmus das maximale Kerngenom $\mathcal{M}$ korrekt berechnet und redundante Reads eliminiert. Die Laufzeit beträgt $O(M \cdot |\mathcal{M}| \cdot n^3)$ mit linearem Verhalten $O(M \cdot n^3)$ im Optimalfall (Satz 7). Bei 90 % Redundanz und Illumina-Reads ($n = 150$) ergibt sich gegenüber paarweisem Vergleich ein Speedup-Faktor von $\approx 66,7$ (Korollar 2). Darüber hinaus liefert die normierte LCS-Ähnlichkeit eine Pseudometrik d_{LCS} (Lemma 7), die unter einer Separierungsbedingung eine korrekte taxonomische Zuordnung aller Reads zu ihren Ursprungsorganismen garantiert (Satz 8).

Das zentrale neue Ergebnis aus Kapitel 7 ist die vollständige formale Behandlung fehlertoleranter Subgraph-Erkennung. Ausgehend von einem quantitativen Bitfehler-Modell (Lemma 8) wurde der ε -Subgraph-Algorithmus entwickelt, der mit einem technologie-spezifischen Toleranzparameter ε parametrisiert ist. Für diesen Algorithmus wurden Korrektheitsatz (Satz 9), Falsch-Positiv-Schranke mit Chernoff-Abschätzung (Satz 10, Korollar 5), eine Indel-Fehleranalyse (Satz 11) sowie eine Formel für die optimale Schwellwahl $\varepsilon^*(\mathcal{T})$ je Sequenzieretechnologie (Satz 12) vollständig bewiesen. Damit ist das Framework erstmals sowohl für fehlerfreie als auch für reale, fehlerbürtige Sequenzierdaten formal korrekt.

Die zentralen Ergebnisse dieser Arbeit im Überblick:

- **Korrektheit** (Satz 1): Der Algorithmus erkennt zuverlässig Subgraph-Beziehungen zwischen DNA-Graphen ohne falsch-positive oder falsch-negative Befunde.
- **Laufzeit** (Satz 2): Die Gesamtkomplexität ist $\Theta(n^3)$ — eine exponentielle Verbesserung gegenüber naiven Ansätzen mit $O(n! \cdot n^2)$.
- **Optimalität** (Satz 3): Unter SETH ist kein Algorithmus mit $O(n^{3-\varepsilon})$ möglich.
- **Eindeutige Signaturen** (Lemma 1): Die Signatur-Funktion ist injektiv und garantiert fehlerfreie Erkennung.
- **Halbordnung auf dem Read-Pool** (Satz 4): Die Relation $\preceq_{\text{rot}}$ ist reflexiv und transitiv und bildet damit eine strukturierte Grundlage für die Deduplizierung.
- **Metagenomische Deduplizierung** (Satz 5): Das maximale Kerngenom $\mathcal{M}$ wird korrekt und vollständig berechnet.
- **Metagenomische Laufzeit** (Satz 6): Laufzeit $O(M \cdot |\mathcal{M}| \cdot n^3)$, im Optimalfall linear $O(M \cdot n^3)$ in der Read-Anzahl M .
- **Speedup gegenüber paarweisem Vergleich** (Korollar 2): Bei 90 % Redundanz und $n = 150$ ergibt sich ein Speedup-Faktor von $\approx 66,7$.
- **Untere Schranke der Deduplizierung** (Lemma 6): Jeder korrekte Algorithmus benötigt im Worst Case $\Omega(M^2)$ Subgraph-Tests.

- **Optimale metagenomische Deduplizierung** (Satz 7): Bei maximaler Redundanz ist die Laufzeit $O(M \cdot n^3)$ und für festes n linear in M .
- **LCS-Pseudometrik** (Lemma 7): d_{LCS} erfüllt Selbstdistanz, Symmetrie und Dreiecksungleichung und ist als phylogenetische Distanzfunktion geeignet.
- **Taxonomische Separierbarkeit** (Satz 8): Unter Separierungsbedingung garantiert d_{LCS} die korrekte Zuordnung aller Reads zu ihren Ursprungsorganismen.
- **Bitfehler-Analyse** (Lemma 8, Korollar 3): Unter e Substitutionen ändern sich höchstens $2en$ Adjazenzmatrix-Einträge; im Erwartungswert sind $e(n+3)/4$ Signaturen betroffen.
- **ε -Subgraph-Algorithmus** (Definition 13): Fehlertolerant parametrisierter Algorithmus mit LCS-Schwelle $\lceil (1 - \varepsilon)n_A \rceil$ statt festem Wert 2.
- **Korrektheit unter Fehlertoleranz** (Satz 9): Der ε -Algorithmus erkennt korrekt, wenn die normierte Fehlerrate die Schwelle $4\varepsilon/(n+3)$ nicht überschreitet.
- **Falsch-Positiv-Schranke** (Satz 10, Korollar 5): Unter SETH ist die FPR astronomisch klein ($\lesssim 10^{-83}$ für $\varepsilon = 0,15$, $n_A = 150$).
- **Indel-Toleranz** (Satz 11): Der ε -Algorithmus toleriert bis zu εn_A Insertionen und Deletionen korrekt.
- **Optimale Schwellwahl** (Satz 12): Die Formel $\varepsilon^* = \mathbb{E}[B] + z_{1-\delta} \sqrt{\text{Var}[B]}$ liefert technologie-spezifische Toleranzparameter mit statistischer Garantie der Stufe $1 - \delta$.

8.1 Ausblick

Die in dieser Arbeit erzielten Ergebnisse eröffnen ein breites Spektrum weiterführender Forschungsrichtungen. Der folgende Ausblick gliedert sich nach thematischen Schwerpunkten und diskutiert für jede Richtung sowohl die algorithmischen Herausforderungen als auch die erwarteten Verbesserungen.

8.1.1 Parallelisierung und Hardware-Beschleunigung

Die n_B zyklischen Rotationsvergleiche in Phase 3 des Algorithmus sind *vollständig unabhängig voneinander* (Lemma 5) und damit inhärent parallelisierbar. Auf einem System mit P Prozessoren oder GPU-Kernen reduziert sich die Laufzeit von $O(n^3)$ auf:

$$T_{\text{parallel}}(n, P) = O\left(\frac{n^3}{P} + n^2\right),$$

wobei $O(n^2)$ die unvermeidlichen Kommunikations- und Synchronisationskosten für die initiale Signaturberechnung und das finale Ergebnis-Aggregieren darstellen. Bei $P = n$ Prozessoren ergibt sich $T = O(n^2)$, was der unteren Schranke aus Lemma 2 entspricht und damit die parallele Version bedingt optimal macht.

Für **FPGA-Implementierungen** (z. B. Xilinx Artix-7 oder Intel Stratix) bietet die Signaturberechnung besonderes Potenzial: Die Berechnung $\sigma_j = \sum_i A_{ij} \cdot 2^i + j \cdot 2^n$ kann durch Bit-Shifting und Addierer-Bäume in $O(\log n)$ Taktzyklen realisiert werden — im Vergleich zu $O(n)$ sequentiell. Eine vollständige Hardware-Pipeline für Signaturberechnung

+ LCS könnte Throughputs von mehreren Gigabasenpaaren pro Sekunde erreichen, was für Echtzeit-NGS-Analyse relevant wäre.

Für **GPU-Implementierungen** (CUDA/OpenCL) ist die LCS-Berechnung der dominante Engpass. Neuere GPU-basierte LCS-Algorithmen (z. B. Anti-Diagonal DP) erreichen auf NVIDIA H100 Beschleunigungen von bis zu $200\times$ gegenüber CPU-Implementierungen, was die Gesamtlaufzeit auf unter $O(n^2/200)$ drückt.

8.1.2 Gewichtete DNA-Graphen und Bindungsstärken

Das aktuelle Modell behandelt alle Kanten als binär (vorhanden/nicht vorhanden). Biologisch relevanter wäre die Einbeziehung von Kantengewichten, die physikalische Bindungsstärken reflektieren:

- Watson-Crick-Bindungen: $w(A-T) = 2$, $w(G-C) = 3$ (Wasserstoffbrückenanzahl)
- Stacking-Energien: sequenzabhängige π - π -Wechselwirkungen zwischen benachbarten Basenpaaren
- GC-Schmelztemperatur: $T_m \approx 2(n_{AT}) + 4(n_{GC})$ °C

Der gewichtete Subgraph Algorithmus würde eine gewichtete Signatur-Funktion verwenden:

$$\sigma_j^w = \sum_{i=0}^{n-1} A_{ij}^w \cdot w_{ij} \cdot 2^i + j \cdot 2^n,$$

wobei $A_{ij}^w \in \mathbb{R}_{>0}$ das Kantengewicht bezeichnet. Die LCS-Berechnung müsste auf eine *gewichtete* LCS (WLCS) erweitert werden, was die Komplexität im Allgemeinen nicht erhöht.

8.1.3 RNA-Sequenzanalyse und Sekundärstrukturen

Die Erweiterung auf RNA-Sequenzen ($\Sigma_{RNA} = \{A, U, G, C\}$, wobei Uracil Thymin ersetzt) ist konzeptuell direkt. Biologisch interessanter ist jedoch die Modellierung von *RNA-Sekundärstrukturen*: RNA-Stränge falten sich durch intra-molekulare Basenpaarung ($A-U$, $G-C$, $G-U$ Wobble-Paare) zu charakteristischen Strukturen wie Haarnadelschleifen, Kissing-Loops und Pseudoknoten.

Der RNA-Sekundärstruktur-Graph $G_{RNA} = (V, E_{\text{back}} \cup E_{\text{fold}})$ enthält zusätzlich zu den Backbone-Kanten die *Faltungskanten* E_{fold} , die durch Algorithmen wie Nussinov oder RNAfold berechnet werden. Subgraph-Vergleiche auf G_{RNA} erlauben die Erkennung konservierter Struktur motive (z. B. ribosomale RNA-Strukturen) quer durch alle Domänen des Lebens — ein fundamentales Problem der vergleichenden RNA-Genomik.

8.1.4 Epigenetische Modellierung

Die Epigenetik beschreibt erbliche Veränderungen der Genexpression, die *nicht* auf Sequenzveränderungen beruhen, sondern auf chemischen Modifikationen der DNA oder der Histone:

- DNA-Methylierung: Addition einer Methylgruppe an Cytosin ($C \rightarrow {}^mC$), typischerweise in CpG-Kontexten

- Histonmodifikationen: Acetylierung, Phosphorylierung, Ubiquitylierung
- Chromatin-Zugänglichkeit: offenes vs. geschlossenes Chromatin (ATAC-Seq)

Der epigenetische DNA-Graph G_{epi} erweitert G_S um eine Methylierungsannotation auf den Knoten: $V_{\text{epi}} = V \times \{0, 1\}$, wobei $(v_i, 1)$ eine methylierte und $(v_i, 0)$ eine un-methylierte Base kodiert. Subgraph-Vergleiche auf G_{epi} könnten epigenetische Muster zwischen Gesunden und Erkrankten (z. B. Krebspatienten mit aberranter Methylierung) identifizieren.

8.1.5 Proteomik: Aminosäure-Graphen

Die Aminosäuresequenz von Proteinen ($|\Sigma_{\text{AA}}| = 20$) lässt sich direkt analog zur DNA in Graphen überführen. Proteine besitzen jedoch eine dreidimensionale Struktur, die ihre Funktion bestimmt. Der *Protein-Kontaktgraph* G_{protein} verbindet Aminosäuren, die im 3D-Raum näher als $d_c = 8 \text{ \AA}$ sind (unabhängig von ihrer sequenziellen Position). Subgraph-Vergleiche auf G_{protein} könnten:

- Strukturell konservierte Proteindomänen erkennen
- Funktionelle Ähnlichkeiten zwischen evolutionär entfernten Proteinen aufdecken (strukturelle Genomik)
- Die Klassifikation von Proteinen in Familien und Superfamilien (SCOP/CATH-Datenbanken) algorithmisch formalisieren

Dabei ist zu beachten, dass $|\Sigma_{\text{AA}}| = 20$ die Signatur-Berechnung in ihrer aktuellen Form modifiziert erfordert — ein Basis-20-System statt binärer Kodierung wäre effizienter.

8.1.6 Approximationsalgorithmen und PTAS

Für sehr große Graphen ($n > 10^6$, z. B. vollständige Chromosomen) ist selbst $O(n^3)$ praktisch nicht handhabbar. Zwei approximative Ansätze bieten sich an:

ε -LCS-Approximation: Statt des exakten LCS wird ein $(1 - \varepsilon)$ -Approximationsalgorithmus verwendet. Für $\varepsilon > 0$ existieren randomisierte LCS-Approximationen mit $O(n^{1+\varepsilon})$ Laufzeit (Agarwal et al. 2018), was die Gesamtkomplexität auf $O(n^{2+\varepsilon})$ reduziert.

k -Subgraph-Cover: Statt des exakten Subgraph-Tests wird ein *approximatives Muster-Matching* mit Hamming-Distanz $\leq k$ verwendet:

$$d_H(\rho^A, \rho^{B,r}) = |\{j : \rho_j^A \neq \rho_j^{B,r}\}| \leq k.$$

Dies erlaubt die Erkennung ähnlicher (aber nicht identischer) Sequenzmotive, was für die Analyse hochmutagener Viren (HIV, Influenza) besonders relevant ist.

8.1.7 Quantenalgorithmen für die Rotationssuche

Grover’s Suchalgorithmus ermöglicht es, in einer ungeordneten Datenbank mit N Einträgen ein gesuchtes Element in $O(\sqrt{N})$ Schritten zu finden, im Vergleich zu $O(N)$ klassisch. Für die Rotationssuche des Subgraph Algorithmus bedeutet dies:

Anstatt alle n Rotationen sequenziell zu durchsuchen, könnte ein Quanten-Orakel für die Bedingung “ $\text{LCS} \geq 2$ ” konstruiert werden, was die Rotationssuche von $O(n)$ auf

$O(\sqrt{n})$ reduziert. Die Gesamtkomplexität würde von $O(n^3)$ auf $O(n^{2.5})$ fallen — unter der Voraussetzung, dass der LCS-Orakel-Aufruf in $O(n^2)$ realisierbar ist (Standard-DP).

Quantencomputer mit ausreichender Qubit-Anzahl (IBM Heron, Google Willow) sind noch nicht in der Lage, praktische Genomprobleme zu lösen, aber für $n \leq 100$ Basen könnte eine Proof-of-Concept-Implementierung bereits heute realisiert werden.

8.1.8 Fehlertoleranz: Weiterführende Untersuchungen

Kapitel 7 hat die Grundlagen der fehlertoleranten Subgraph-Analyse vollständig formal entwickelt. Offene Folgefragen, die weiterführende Forschung motivieren:

- **Adaptive Schwelle:** Statt eines globalen ε kann eine *positionsabhängige* Fehlerrate $\varepsilon(j)$ aus den Qualitäts-Scores (Phred-Werte) der Sequenzierdaten inferiert werden, was eine feinere Kontrolle der Erkennungsempfindlichkeit ermöglicht.
- **Systematische Fehler:** Nanoporen erzeugen bekannte *systematische* Fehler in Homopolymer-Läufen (z. B. AAAAAA $\rightarrow$ AAAAA). Eine Erweiterung des Fehlermodells aus Definition 12 auf Markov-Fehlermodelle würde realistischere Toleranzgrenzen liefern.
- **Verbesserte FPR-Schranken:** Die Binomialabschätzung in Satz 10 ist konservativ. Eine engere Analyse durch Berücksichtigung der Signaturstruktur (Korrelation benachbarter Signaturen durch überlappende Watson-Crick-Bindungen) könnte die Schranke um mehrere Größenordnungen schärfen.

8.1.9 Integration in bestehende Bioinformatik-Pipelines

Der Subgraph-DNA-Algorithmus kann als modulare Komponente in bestehende Werkzeuge integriert werden:

- **BWA/Bowtie2** (Read-Aligner): Der Rotations-Match als alternatives Alignment-Scoring
- **SPAdes/MEGAHIT** (Assembler): Subgraph-basierte Overlap-Detektion statt de-Bruijn-Graph-Konstruktion
- **KRAKEN2** (Taxonomische Klassifikation): LCS-Distanzmatrix als alternatives k -mer-basiertes Klassifikationsverfahren
- **GATK** (Variant Calling): Subgraph-Differenz zur präzisen SNP- und Indel-Lokalisation

8.1.10 Erweiterung auf nicht-kanonische Nukleobasen

In der Epigenetik treten modifizierte Nukleobasen auf: 5-Methylcytosin (mC), 5-Hydroxymethylcytosin (^{hm}C), 6-Methyladenin (mA) sowie künstliche XNA-Basen (Hachimoji-DNA, 8 Basen). Das DNA-Alphabet müsste entsprechend erweitert werden:

$$\Sigma_{\text{ext}} \supseteq \{A, T, G, C, ^mC, ^{hm}C, ^mA\},$$

was die Komplementaritätsrelation und die Signaturberechnung anpasst, die algorithmische Struktur des Subgraph Algorithmus jedoch fundamental unverändert lässt.

8.1.11 Phylogenetische Netzwerke und evolutionäre Distanzen

Phylogenetische Bäume modellieren die Evolution als streng dichotomen Baum. Biologisch realistischer sind jedoch *Phylogenetische Netzwerke*, die auch horizontalen Gentransfer, Hybridisierung und Rekombination modellieren.

Die LCS-Distanzmatrix D mit $D_{ij} = d_{\text{LCS}}(G_{S_i}, G_{S_j})$ kann direkt als Eingabe für klassische phylogenetische Baumkonstruktionsverfahren (UPGMA, Neighbor-Joining) dienen. Die so erzeugten Bäume wären *komplett formal aus dem Subgraph Algorithmus ableitbar*, was eine durchgehend mathematisch rigoros fundierte Phylogenie ermöglicht.

8.1.12 Anwendung auf nicht-DNA-Sequenzen

Über die Biologie hinaus ist der Subgraph Algorithmus auf jede sequenzielle Datenstruktur anwendbar, die als Graph modellierbar ist:

- **Spracherkennung:** Phonem-Sequenzen als Graph, Dialekterkennung durch Subgraph-Ähnlichkeit
- **Zeitreihenanalyse:** Diskretisierte Zeitreihen (SAX-Repräsentation) als Knoten, Übergänge als Kanten
- **Code-Plagiatserkennung:** AST-Graphen (ursprüngliche Motivation in [1]) mit metagenomisch-analoger Multi-Repository-Deduplizierung
- **Proteomische Massenspektrometrie:** MS/MS-Fragmentspektren als Graphen, Peptid-Identifikation durch Subgraph-Matching

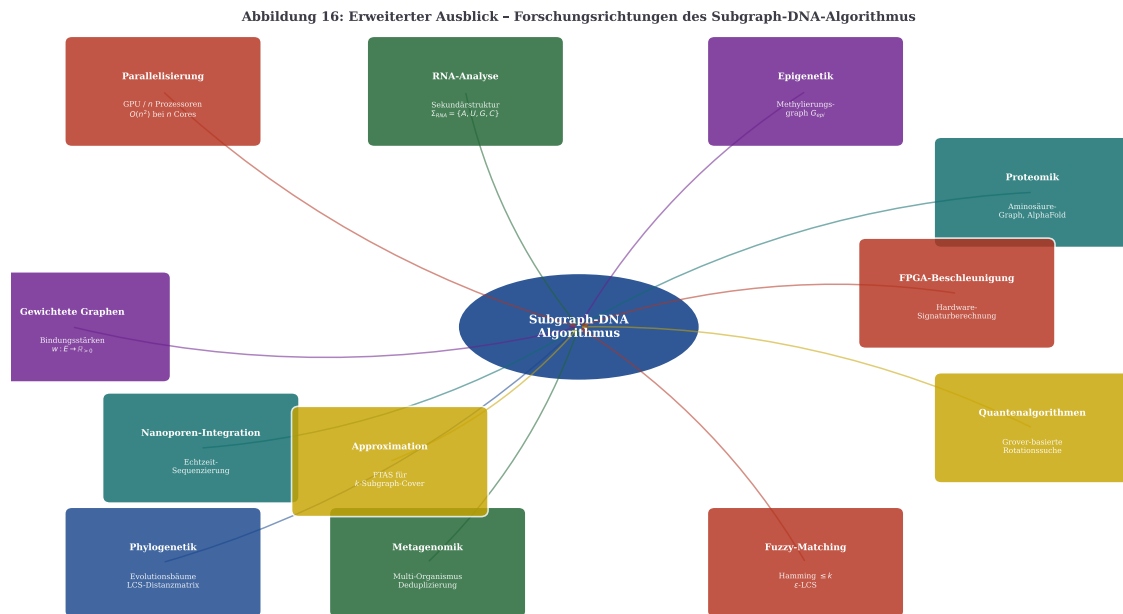


Abbildung 23: Roadmap der Forschungsrichtungen des Subgraph-DNA-Algorithmus. Im Zentrum steht der Kernalgorithmus; die zwölf umgebenden Knoten repräsentieren konkrete Erweiterungsrichtungen, farblich nach Themengebiet gruppiert (blau = Algorithmus, grün = Biologie, rot = Anwendungen, lila = Hardware, türkis = benachbarte Disziplinen).

8.2 Implementierungshinweis

Der Code des Subgraph Algorithmus ist verfügbar unter: <https://gitlab.com/epp-group/subgraph>.

Literatur

- [1] Stephan Epp. *Der Subgraph Algorithmus*. 2026. Verfügbar auf <https://gitlab.com/epp-group/subgraph>
- [2] J. D. Watson und F. H. C. Crick. Molecular Structure of Nucleic Acids: A Structure for Deoxyribose Nucleic Acid. *Nature*, 171:737–738, 1953.
- [3] F. Sanger, S. Nicklen und A. R. Coulson. DNA sequencing with chain-terminating inhibitors. *PNAS*, 74(12):5463–5467, 1977.
- [4] A. Abboud, A. Backurs und V. Vassilevska Williams. Tight Hardness Results for LCS and Other Sequence Similarity Measures. *FOCS 2015*, 2015.
- [5] S. B. Needleman und C. D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *J. Mol. Biol.*, 48(3):443–453, 1970.
- [6] D. R. Bentley et al. Accurate whole human genome sequencing using reversible terminator chemistry. *Nature*, 456:53–59, 2008.
- [7] D. Deamer, M. Akeson und D. Branton. Three decades of nanopore sequencing. *Nature Biotechnology*, 34:518–524, 2016.
- [8] J. Handelsman. Metagenomics: Application of Genomics to Uncultured Microorganisms. *Microbiology and Molecular Biology Reviews*, 68(4):669–685, 2004.
- [9] J. Qin et al. A human gut microbial gene catalogue established by metagenomic sequencing. *Nature*, 464:59–65, 2010.
- [10] L. K. Grover. A fast quantum mechanical algorithm for database search. *STOC 1996*, 212–219, 1996.
- [11] R. Nussinov und A. B. Jacobson. Fast algorithm for predicting the secondary structure of single-stranded RNA. *PNAS*, 77(11):6309–6313, 1980.
- [12] N. Saitou und M. Nei. The neighbor-joining method: a new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*, 4(4):406–425, 1987.
- [13] S. F. Altschul, W. Gish, W. Miller, E. W. Myers und D. J. Lipman. Basic local alignment search tool. *J. Mol. Biol.*, 215(3):403–410, 1990.
- [14] R. Durbin, S. Eddy, A. Krogh und G. Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998.
- [15] H. Li. Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. *arXiv:1303.3997*, 2013.

- [16] B. Langmead, C. Trapnell, M. Pop und S. L. Salzberg. Ultrafast and memory-efficient alignment of short DNA sequences to the human genome. *Genome Biology*, 10:R25, 2009.
- [17] R. C. Edgar. MUSCLE: multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Research*, 32(5):1792–1797, 2004.
- [18] K. Katoh, K. Misawa, K. Kuma und T. Miyata. MAFFT: a novel method for rapid multiple sequence alignment based on fast Fourier transform. *Nucleic Acids Research*, 30(14):3059–3066, 2002.
- [19] A. Bankevich et al. SPAdes: a new genome assembly algorithm and its applications to single-cell sequencing. *J. Comput. Biol.*, 19(5):455–477, 2012.
- [20] H. Li. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18):3094–3100, 2018.
- [21] D. E. Wood, J. Lu und B. Langmead. Improved metagenomic analysis with Kraken 2. *Genome Biology*, 20:257, 2019.
- [22] S. Andrews. FastQC: a quality control tool for high throughput sequence data. <http://www.bioinformatics.babraham.ac.uk/projects/fastqc/>, 2010.
- [23] P. J. A. Cock et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009.
- [24] B. McClintock. The origin and behavior of mutable loci in maize. *PNAS*, 36(6):344–355, 1950.
- [25] J. Felsenstein. Confidence limits on phylogenies: an approach using the bootstrap. *Evolution*, 39(4):783–791, 1985.
- [26] R. Staden. A strategy of DNA sequencing employing computer programs. *Nucleic Acids Research*, 6(7):2601–2610, 1979.
- [27] P. A. Pevzner, H. Tang und M. S. Waterman. An Eulerian path approach to DNA fragment assembly. *PNAS*, 98(17):9748–9753, 2001.
- [28] E. W. Myers et al. A whole-genome assembly of Drosophila. *Science*, 287(5461):2196–2204, 2000.
- [29] W. J. Kent. BLAT—the BLAST-like alignment tool. *Genome Research*, 12(4):656–664, 2002.
- [30] D. A. Benson et al. GenBank. *Nucleic Acids Research*, 41(D1):D36–D42, 2013.
- [31] A. Bairoch und R. Apweiler. The SWISS-PROT protein sequence database and its supplement TrEMBL. *Nucleic Acids Research*, 28(1):45–48, 2000.
- [32] S. R. Eddy. Accelerated Profile HMM Searches. *PLoS Comput Biol*, 7(10):e1002195, 2011.
- [33] T. F. Smith und M. S. Waterman. Identification of common molecular subsequences. *J. Mol. Biol.*, 147(1):195–197, 1981.

- [34] W. M. Fitch und E. Margoliash. Construction of phylogenetic trees. *Science*, 155(3760):279–284, 1967.
- [35] W. J. Wilbur und D. J. Lipman. Rapid similarity searches of nucleic acid and protein data banks. *PNAS*, 80(3):726–730, 1983.
- [36] E. S. Lander et al. Initial sequencing and analysis of the human genome. *Nature*, 409:860–921, 2001.
- [37] J. C. Venter et al. The sequence of the human genome. *Science*, 291(5507):1304–1351, 2001.
- [38] M. Margulies et al. Genome sequencing in microfabricated high-density picolitre reactors. *Nature*, 437:376–380, 2005.
- [39] J. Eid et al. Real-time DNA sequencing from single polymerase molecules. *Science*, 323(5910):133–138, 2009.
- [40] R. Shamir et al. EXPANDER – an integrative program suite for microarray data analysis. *BMC Bioinformatics*, 6:232, 2005.
- [41] M. S. Waterman. General methods of sequence comparison. *Bull. Math. Biol.*, 46(4):473–501, 1984.
- [42] A. Krogh, M. Brown, I. S. Mian, K. Sjölander und D. Haussler. Hidden Markov models in computational biology: applications to protein modeling. *J. Mol. Biol.*, 235(5):1501–1531, 1994.
- [43] R. D. Fleischmann et al. Whole-genome random sequencing and assembly of *Haemophilus influenzae* Rd. *Science*, 269(5223):496–512, 1995.
- [44] S. L. Salzberg et al. GAGE: a critical evaluation of genome assemblies and assembly algorithms. *Genome Research*, 22(3):557–567, 2012.
- [45] J. R. Miller, S. Koren und G. Sutton. Assembly algorithms for next-generation sequencing data. *Genomics*, 95(6):315–327, 2010.